ĐỀ CƯƠNG ÔN TẬP
Học phần: CẤU TRÚC DỮ LIỆU – GIẢI THUẬT

Chương 1: Giải thuật và phân tích giải thuật

Bài 1: Viết thuật toán giả để quay một số điện thoại.

```
Thuật toán QUAY_SO_DIEN_THOAI(S, t, r)
  Cho i ← 0 đến độ dài(S) - 1:
    digit ← S[i]
    Nếu digit == '0' thì
      thời_gian_quay ← 10 * t
    Ngược lại
      thời_gian_quay ← GIÁ_TRỊ_SỐ(digit) * t
    IN("Quay số", digit, "→ Mất", thời_gian_quay, "đơn vị thời gian")
    Nếu i < độ dài(S) - 1 thì
      IN("Nghỉ", r, "đơn vị thời gian trước khi quay số tiếp theo")
  IN("Hoàn tất quay số:", S)
Kết thúc
```

Bài 2: Viết thuật toán giả và hàm để tăng lương cho tất cả nhân viên trong công ty với mức tăng chi phí sinh hoạt 3.2%. Giả sử tệp bảng lương đã có đầy đủ nhân viên hiện tại.

*Giả mã:

```
Thuật toán TANG_LUONG_TOAN_CONG_TY(danh_sach_nhan_vien)
  Tỉ lệ tăng ← 1.032
  Cho mỗi nhân viên trong danh_sach_nhan_vien:
    lương_cũ ← nhân_viên.lương
    lương_mới ← lương_cũ × tỉ lệ tăng
    nhân_viên.lương ← làm tròn lương_mới (nếu cần)
  Trả về danh_sach_nhan_vien đã cập nhật
Kết thúc
```

*Chương trình:

```python
def tang_luong(bang_luong, ty_le_tang=0.032):
    bang_luong_moi = []
    for nhan_vien in bang_luong:
        luong_moi = nhan_vien['luong'] * (1 + ty_le_tang)
        bang_luong_moi.append({
            'ten': nhan_vien['ten'],
            'luong': round(luong_moi, 2)
        })
    return bang_luong_moi
bang_luong = [
    {'ten': 'Nguyen Van A', 'luong': 10000000},
    {'ten': 'Tran Thi B', 'luong': 12000000},
    {'ten': 'Le Van C', 'luong': 9500000}
```

```
]
bang_luong_moi = tang_luong(bang_luong)
for nv in bang_luong_moi:
    print(f"{nv['ten']}: {nv['luong']} VND")
```

Bài 3: Thiết kế hệ thống kiểm kê sách giáo khoa. Bạn được thuê làm thực tập sinh mùa hè để thiết kế một hệ thống kiểm kê sách giáo khoa cho nhà sách của trường. Hệ thống này bao gồm các quy trình chính sau:

a. Đặt sách giáo khoa b. Nhận sách giáo khoa

c. Xác định giá bán lẻ d. Định giá sách cũ

e. Xác định số lượng hiện có f. Ghi nhận việc bán sách g. Ghi nhận việc trả sách

Viết phần khai báo thuật toán (abstract data type) cho các quy trình trên. Mỗi phần khai báo nên bao gồm: tên, tham số, mục đích, điều kiện trước, điều kiện sau, và kiểu giá trị trả về.

a. Đặt sách giáo khoa (DatSach)

Tên: DatSach

Tham số: ten_sach, ma_sach, so_luong, ngay_dat

Mục đích: Tạo đơn đặt hàng sách giáo khoa mới từ nhà xuất bản.

Điều kiện trước: ma_sach chưa tồn tại trong kho hoặc đang hết hàng.

Điều kiện sau: Đơn đặt hàng mới được ghi nhận trong hệ thống.

Giá trị trả về: Đơn hàng (dạng object hoặc ID)

b. Nhận sách giáo khoa (NhanSach)

Tên: NhanSach

Tham số: ma_sach, so_luong_nhan, ngay_nhan

Mục đích: Cập nhật kho khi nhận được sách từ nhà cung cấp.

Điều kiện trước: Đã có đơn đặt sách với ma_sach đang chờ xử lý.

Điều kiện sau: Số lượng sách hiện có tăng thêm so_luong_nhan.

Giá trị trả về: Boolean (True nếu nhận thành công)

c. Xác định giá bán lẻ (XacDinhGiaBanLe)

Tên: XacDinhGiaBanLe

Tham số: ma_sach, gia_goc, ty_le_loi_nhuan

Mục đích: Tính và lưu giá bán lẻ dựa trên giá gốc và lợi nhuận mong muốn.

Điều kiện trước: Sách đã tồn tại trong kho, có gia_goc.

Điều kiện sau: Ghi nhận gia_ban_le = gia_goc * (1 + ty_le_loi_nhuan)

Giá trị trả về: gia_ban_le (kiểu số thực)

d. Định giá sách cũ (DinhGiaSachCu)

Tên: DinhGiaSachCu

Tham số: ma_sach, tinh_trang (ví dụ: mới, tốt, trung bình, kém)

Mục đích: Xác định giá bán lại cho sách cũ dựa vào tình trạng sách.

Điều kiện trước: Sách thuộc danh sách sách cũ trong kho.

Điều kiện sau: Gán gia_sach_cu = gia_ban_le * ty_le_giam

Giá trị trả về: gia_sach_cu (số thực)

e. Xác định số lượng hiện có (SoLuongHienCo)

Tên: SoLuongHienCo

Tham số: ma_sach

Mục đích: Truy vấn số lượng tồn kho hiện tại của một mã sách.

Điều kiện trước: Sách đã có trong kho.
Điều kiện sau: Không thay đổi trạng thái hệ thống.
Giá trị trả về: so_luong_hien_co (kiểu nguyên)

f. Ghi nhận việc bán sách (GhiNhanBanSach)

Tên: GhiNhanBanSach
Tham số: ma_sach, so_luong_ban, ngay_ban, gia_ban
Mục đích: Lưu thông tin giao dịch bán sách.
Điều kiện trước: Số lượng hiện có ≥ so_luong_ban.
Điều kiện sau: Số lượng tồn kho giảm, hóa đơn được ghi nhận.
Giá trị trả về: ID_HoaDon hoặc Boolean thành công/thất bại

g. Ghi nhận việc trả sách (GhiNhanTraSach)

Tên: GhiNhanTraSach
Tham số: ma_sach, so_luong_tra, ngay_tra, ly_do
Mục đích: Ghi lại sách bị trả lại và cập nhật kho.
Điều kiện trước: Sách đã được bán và còn trong thời gian cho phép hoàn trả.
Điều kiện sau: Kho tăng so_luong_tra, ghi nhận lý do.
Giá trị trả về: Boolean (True nếu trả thành công)

Bài 4: Đếm chữ số và tính tổng chữ số của một số nguyên

*Mã giả:

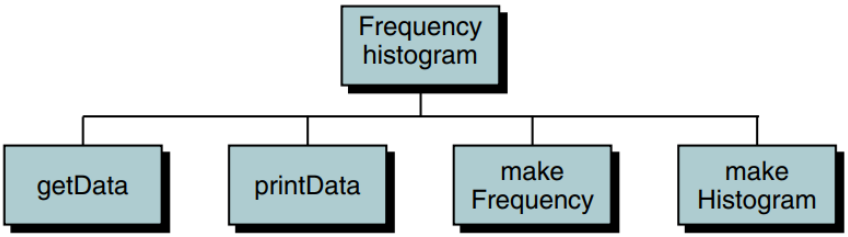
```
Thuật toán DemVaTongChuSo(n: Integer)
  Nếu n < 0 thì
    n ← |n|
  dem ← 0
  tong ← 0
  Nếu n == 0 thì
    dem ← 1
    tong ← 0
  Ngược lại
    Trong khi n > 0 làm:
      chu_so ← n % 10
      tong ← tong + chu_so
      dem ← dem + 1
      n ← n div 10
  Trả về (dem, tong)
Kết thúc
```

*Chương trình:

```python
def dem_va_tong_chu_so(n):
    n = abs(n)
    if n == 0:
        return 1, 0
    dem = 0
    tong = 0
    while n > 0:
```

```
        chu_so = n % 10
        tong += chu_so
        dem += 1
        n //= 10
    return dem, tong
n = int(input("Nhập một số nguyên: "))
so_chu_so, tong_chu_so = dem_va_tong_chu_so(n)
print("Số chữ số:", so_chu_so)
print("Tổng các chữ số:", tong_chu_so)
```

Bài 5: Viết mã giả (pseudocode) cho một chương trình xây dựng một mảng tần suất cho các giá trị dữ liệu trong khoảng từ 1 đến 20 và sau đó in ra biểu đồ tần suất của chúng. Dữ liệu được đọc từ một tập tin. Thiết kế của chương trình được minh họa trong hình bên dưới.



Mỗi thuật toán con được mô tả như sau:

a. Thuật toán *getData* đọc tệp và lưu dữ liệu vào một mảng.

b. Thuật toán *printData* in ra dữ liệu trong mảng.

c. Thuật toán *makeFrequency* kiểm tra dữ liệu trong mảng, từng phần tử một, và cộng thêm 1 vào phần tử tương ứng trong mảng tần suất dựa trên giá trị dữ liệu.

d. Thuật toán *makeHistogram* in ra biểu đồ dạng dọc bằng cách sử dụng dấu sao cho mỗi lần xuất hiện của một phần tử. Ví dụ, nếu có năm số 1 và tám số 2 trong dữ liệu, nó sẽ in:

1: *****

2: ********

*Mã giả:

Thuật toán FrequencyHistogram

1. data ← getData()
2. printData(data)
3. frequency ← makeFrequency(data)
4. makeHistogram(frequency)

a.

Thuật toán getData()

1. Mở tệp dữ liệu để đọc
2. Tạo danh sách rỗng: data ← []
3. Trong khi chưa hết tệp:
   a. Đọc một dòng và chuyển thành số nguyên
   b. Nếu giá trị nằm trong khoảng 1 đến 20:
      i. Thêm giá trị vào data
4. Trả về data

b.

Thuật toán printData(data)

1. In ra chuỗi: "Dữ liệu đã đọc:"
2. Với mỗi phần tử x trong data:
   a. In x cách nhau bởi khoảng trắng
c.
Thuật toán makeFrequency(data)
1. Tạo mảng frequency gồm 20 phần tử, khởi tạo tất cả bằng 0
2. Với mỗi phần tử x trong data:
   a. frequency[x - 1] ← frequency[x - 1] + 1
3. Trả về frequency
d.
Thuật toán makeHistogram(frequency)
1. In chuỗi: "Biểu đồ tần suất:"
2. Với i từ 1 đến 20:
   a. Nếu frequency[i - 1] > 0:
      i. In i và dấu ":"
      ii. In ra frequency[i - 1] dấu *
*Chương trình:

```
def getData(filename):
    data = []
    try:
        with open(filename, 'r') as file:
            for line in file:
                for value in line.strip().split():
                    num = int(value)
                    if 1 <= num <= 20:
                        data.append(num)
    except FileNotFoundError:
        print(f"Tệp {filename} không tồn tại.")
    return data
def printData(data):
    print("Dữ liệu đã đọc:")
    print(" ".join(str(x) for x in data))
def makeFrequency(data):
    frequency = [0] * 20
    for x in data:
        frequency[x - 1] += 1
    return frequency
def makeHistogram(frequency):
    print("\nBiểu đồ tần suất:")
    for i in range(20):
        if frequency[i] > 0:
            print(f"{i + 1}: {'*' * frequency[i]}")
def main():
```

```
    filename = "dulieu.txt"
    data = getData(filename)
    if data:
        printData(data)
        frequency = makeFrequency(data)
        makeHistogram(frequency)
main()
```

Chương 2: Giải thuật và phân tích giải thuật

Bài 1: Xét thuật toán sau:

```
algorithm fun1 (x)
1  if (x < 5)
   1   return (3 * x)
2  else
   1   return (2 * fun1 (x - 5) + 7)
3  end if
end fun1
```

Giá trị trả về sẽ là gì nếu gọi hàm fun1 với các đối số sau:

a. fun1(4)?

b. fun1(10)?

c. fun1(12)?

a.

Vì 4 < 5, nên trả về: fun1(4) = 3 * 4 = 12

b.

Vì 10 >= 5, nên ta dùng công thức đệ quy:

fun1(10) = 2 * fun1(10 - 5) + 7 = 2 * fun1(5) + 7

Tiếp tục tính fun1(5): fun1(5) = 2 * fun1(0) + 7

Tính fun1(0): fun1(0) = 3 * 0 = 0

=> fun1(5) = 2 * 0 + 7 = 7

=> fun1(10) = 2 * 7 + 7 = 21

c.

Vì 12 >= 5, dùng công thức đệ quy:

fun1(12) = 2 * fun1(7) + 7

Tính fun1(7): fun1(7) = 2 * fun1(2) + 7 => fun1(2) = 3 * 2 = 6

=> fun1(7) = 2 * 6 + 7 = 19

=> fun1(12) = 2 * 19 + 7 = 45

Bài 2: Xét thuật toán sau:

```
algorithm fun2 (x, y)
1  if (x < y)
   1   return -3
2  else
   1   return (fun2 (x - y, y + 3) + y)
3  end if
end fun2
```

Giá trị trả về sẽ là gì nếu gọi hàm fun2 với các đối số sau:

a. fun2(2, 7)?

b. fun2(5, 3)?

c. fun2(15, 3)?

a.

Do 2 < 7 nên: fun2(2, 7) = -3

b.

Bước 1: 5 >= 3 → gọi đệ quy:

fun2(5, 3) = fun2(5 - 3, 3 + 3) + 3 = fun2(2, 6) + 3

Bước 2: 2 < 6 → return -3

→ fun2(5, 3) = -3 + 3 = 0

c.

Lặp từng bước:

fun2(15, 3) → fun2(12, 6) + 3

fun2(12, 6) → fun2(6, 9) + 6

fun2(6, 9) → 6 < 9 → return -3

Quay ngược lại:

fun2(6, 9) = -3

fun2(12, 6) = -3 + 6 = 3

fun2(15, 3) = 3 + 3 = 6

Bài 3: Xét thuật toán sau:

```
algorithm fun3 (x, y)
1  if (x > y)
   1   return -1
2  elseif (x equal y)
   1   return 1
3  else
   1   return (x * fun3 (x + 1, y))
4  end if
end fun3
```

Giá trị trả về sẽ là gì nếu gọi hàm fun3 với các đối số sau:

a. fun3 (10,4)?

b. fun3 (4,3)?

c. fun3 (4,7)?

d. fun3 (0,0)?

a.

Do 10 > 4 → return -1

b.

Do 4 > 3 → return -1

c.

Chạy từng bước:

fun3(4,7) = 4 * fun3(5,7)

fun3(5,7) = 5 * fun3(6,7)

fun3(6,7) = 6 * fun3(7,7)

fun3(7,7) = 1 (vì x == y)

=> fun3(6,7) = 6 * 1 = 6

=> fun3(5,7) = 5 * 6 = 30

=> fun3(4,7) = 4 * 30 = 120

d.

Vì x == y, return 1

Bài 4: Một trong những phương pháp để tính căn bậc hai của một số là phương pháp Newton. Công thức của phương pháp Newton được thể hiện như sau. Hãy viết mã giả (pseudocode) cho một thuật toán đệ quy để tính căn bậc hai bằng phương pháp Newton. Xác minh thuật toán của bạn bằng cách tính thủ công với các test sau:

- squareRoot(5, 2, 0.01)
- squareRoot(4, 2, 0.01)

Lưu ý: Trong công thức, tol là viết tắt của *tolerance* – tức là độ sai số cho phép.

$$\text{squareRoot}(num, ans, tol) = \begin{cases} ans & \text{if } |ans^2 - num| \le tol \\ \text{squareRoot}(num, (ans^2 + num)/(2 \times ans), tol) & \text{otherwise} \end{cases}$$

*Mã giả:

Thuật toán squareRoot(num, ans, tol)

1. Nếu |ans^2 - num| ≤ tol:
   a. Trả về ans
2. Ngược lại:
   a. Gọi đệ quy: squareRoot(num, (ans^2 + num) / (2 * ans), tol)

*Chương trình:

```
def squareRoot(num, ans, tol):
    if abs(ans**2 - num) <= tol:
        return ans
    else:
        new_ans = (ans**2 + num) / (2 * ans)
        return squareRoot(num, new_ans, tol)
print("sqrt(5) ≈", round(squareRoot(5, 2, 0.01), 4))
print("sqrt(4) ≈", round(squareRoot(4, 2, 0.01), 4))
```

*Chạy thủ công:

Test 1: squareRoot(5, 2, 0.01)

Ta bắt đầu với ans = 2

Lần	ans	ans^2	$ans^2 - 5$
1	2.000	4.000	1.000
2	2.250	5.063	0.063
3	2.236	5.000	0.000 (dừng lại)

→ Kết quả gần đúng: **2.236**

Test 2: squareRoot(4, 2, 0.01)

Lần	ans	ans^2	$ans^2 - 5$
1	2.000	4.000	0.000 (dừng)

→ Kết quả: **2.0**

Bài 5: Sự kết hợp của n đối tượng, chẳng hạn như những quả bóng trong một cái giỏ, được lấy k đối tượng một lúc có thể được tính toán một cách đệ quy bằng cách sử dụng công thức được hiển thị trong Hình 2-16. Đây là một công thức hữu ích. Ví dụ, một số xổ số của các bang yêu cầu người chơi chọn sáu số từ một loạt các số có thể có. Công thức này có thể được sử dụng để tính số lượng các tổ hợp có thể có, k, của n đối tượng. Ví dụ, với 49 số, có C(49,6), hay 13,983,816, tổ hợp khác nhau của sáu số. Viết một thuật toán đệ quy để tính toán tổ hợp của n đối tượng lấy k đối tượng một lúc.

$$C(n, k) = \begin{cases} 1 & \text{if } k = 0 \text{ or } n = k \\ C(n, k) = C(n-1, k) + C(n-1, k-1) & \text{if } n > k > 0 \end{cases}$$

*Mã giả:

Thuật toán combination(n, k)

1. Nếu k = 0 hoặc k = n:
   a. Trả về 1
2. Ngược lại nếu n > k > 0:
   a. Trả về combination(n - 1, k) + combination(n - 1, k - 1)

*Chương trình:

```
def combination(n, k):
    if k == 0 or k == n:
        return 1
    else:
        return combination(n - 1, k) + combination(n - 1, k - 1)
print("C(5, 2) =", combination(5, 2))       # 10
print("C(6, 3) =", combination(6, 3))       # 20
print("C(49, 6) =", combination(49, 6))     # 13983816
```

Bài 6: Số Ackermann, được sử dụng trong logic toán học, có thể được tính toán bằng cách sử dụng công thức được hiển thị bên dưới. Viết một thuật toán đệ quy để tính số Ackermann. Kiểm tra thuật toán của bạn bằng cách sử dụng nó để tính toán thủ công các trường hợp kiểm thử sau: Ackerman(2, 3), Ackerman(2, 5), Ackerman(0, 3), và Ackerman(3, 0).

$$\text{Ackerman}(m,\ n) = \begin{cases} n + 1 & \text{if } m = 0 \\ \text{Ackerman}(m - 1, 1) & \text{if } n = 0 \text{ and } m > 0 \\ \text{Ackerman}(m - 1, \text{Ackerman}(m, n - 1)) & \text{otherwise} \end{cases}$$

*Mã giả:

Thuật toán Ackermann(m, n):

1. Nếu m == 0:
   Trả về n + 1
2. Nếu n == 0 và m > 0:
   Trả về Ackermann(m - 1, 1)
3. Ngược lại:
   Trả về Ackermann(m - 1, Ackermann(m, n - 1))

*Chương trình:

```
def ackermann(m, n):
    if m == 0:
        return n + 1
    elif n == 0:
        return ackermann(m - 1, 1)
    else:
        return ackermann(m - 1, ackermann(m, n - 1))
```

*Chạy thủ công:

+) Ackermann(0, 3): m = 0 → kết quả là n + 1 = 3 + 1 = 4

+) Ackermann(3, 0):

Ack(3, 0) = Ack(2, 1) Ack(2, 1) = Ack(1, Ack(2, 0))

Ack(2, 0) = Ack(1, 1) Ack(1, 1) = Ack(0, Ack(1, 0))

Ack(1, 0) = Ack(0, 1) = 2 → Ack(1, 1) = Ack(0, 2) = 3 → Ack(2, 0) = 3 → Ack(2, 1) = Ack(1, 3)

Tiếp tục:

Ack(1, 3) = Ack(0, Ack(1, 2))

Ack(1, 2) = Ack(0, Ack(1, 1)) = Ack(0, 3) = 4

→ Ack(1, 3) = Ack(0, 4) = 5

→ Ack(3, 0) = **5**

+) Ackermann(2, 3):

Ack(2, 3) = Ack(1, Ack(2, 2))

Ack(2, 2) = Ack(1, Ack(2, 1))

Ack(2, 1) = Ack(1, Ack(2, 0))

Ack(2, 0) = Ack(1, 1)

Ack(1, 1) = Ack(0, Ack(1, 0))

Ack(1, 0) = Ack(0, 1) = 2

→ Ack(1, 1) = Ack(0, 2) = 3

→ Ack(2, 0) = 3 → Ack(2, 1) = Ack(1, 3)

Ack(1, 3) = Ack(0, Ack(1, 2))

Ack(1, 2) = Ack(0, Ack(1, 1)) = Ack(0, 3) = 4 → Ack(1, 3) = Ack(0, 4) = 5

→ Ack(2, 1) = 5 → Ack(2, 2) = Ack(1, 5)

Ack(1, 5) = Ack(0, Ack(1, 4)) = Ack(0, Ack(0, Ack(1, 3))) = …

→ Sau cùng Ack(2, 3) = **9**

+) Ackermann(2, 5): Ack(2, 5) = 13

Bài 7: Viết một thuật toán đệ quy tính toán và trả về độ dài của một danh sách.

*Mã giả:

Hàm length(list):

  Nếu list là rỗng:

    Trả về 0

  Ngược lại:

    Trả về 1 + length(phần còn lại của list)

*Chương trình:

```
def length(lst):
    if lst == []:
        return 0
    else:
        return 1 + length(lst[1:])
print(length([3, 7, 2, 9]))   # Output: 4
print(length([]))             # Output: 0
print(length(['a', 'b']))     # Output: 2
```

Bài 8: Viết một thuật toán đệ quy chuyển đổi một chuỗi các chữ số thành một số nguyên. Ví dụ, "43567" sẽ được chuyển đổi thành 43567.

*Mã giả:

Hàm stringToInt(s):

  Nếu độ dài của s == 1:

    Trả về giá trị số nguyên của ký tự đầu tiên

  Ngược lại:

    Trả về stringToInt(phần chuỗi s bỏ ký tự cuối) * 10 + giá trị số nguyên của ký tự cuối

*Chương trình:

```
def stringToInt(s):
    if len(s) == 1:
        return ord(s[0]) - ord('0')
    else:
        return stringToInt(s[:-1]) * 10 + (ord(s[-1]) - ord('0'))
print(stringToInt("43567"))  # Output: 43567
print(stringToInt("0"))      # Output: 0
print(stringToInt("123"))    # Output: 123
```

Bài 9: Viết một thuật toán đệ quy để cộng n phần tử đầu tiên của chuỗi 1+1/2+1/3+1/4+1/5+⋯+1/n

*Mã giả:

Hàm harmonicSum(n):

  Nếu n == 1:

    Trả về 1

  Ngược lại:

    Trả về 1/n + harmonicSum(n - 1)

*Chương trình:

```
def harmonicSum(n):
    if n == 1:
        return 1
    else:
        return 1 / n + harmonicSum(n - 1)
print(harmonicSum(1))   # Output: 1.0
print(harmonicSum(2))   # Output: 1.5
print(harmonicSum(5))   # Output: ≈ 2.28333
print(harmonicSum(10))  # Output: ≈ 2.92897
```

Bài 10: Viết một thuật toán đệ quy để xác định xem một chuỗi có phải là chuỗi đối xứng (palindrome) hay không. Một chuỗi là chuỗi đối xứng nếu nó có thể được đọc từ phía trước và phía sau với cùng một ý nghĩa. Viết hoa và khoảng trắng sẽ bị bỏ qua. Ví dụ, "anna" và "go dog" là các chuỗi đối xứng. Kiểm tra thuật toán của bạn với hai chuỗi đối xứng sau và ít nhất một trường hợp không phải chuỗi đối xứng: "Madam, I'm Adam"; "Able was I ere I saw Elba".

*Mã giả:

Hàm isPalindrome(s):

  Làm sạch s bằng cách:

    - Bỏ khoảng trắng

- Bỏ ký tự không phải chữ cái hay số
- Chuyển sang chữ thường

Nếu độ dài của s <= 1:

Trả về True

Nếu ký tự đầu khác ký tự cuối:

Trả về False

Ngược lại:

Trả về isPalindrome(s[1:-1]) // bỏ ký tự đầu và cuối

*Chương trình:

```
def cleanString(s):
    result = ""
    for c in s:
        # Kiểm tra nếu là chữ cái hoặc chữ số
        if c.isalnum():
            # Nếu là chữ hoa, chuyển sang chữ thường
            if 'A' <= c <= 'Z':
                result += chr(ord(c) + 32)
            else:
                result += c
    return result
def isPalindrome(s):
    s = cleanString(s)
    if len(s) <= 1:
        return True
    if s[0] != s[-1]:
        return False
    return isPalindrome(s[1:-1])
print(isPalindrome("Madam, I’m Adam"))             # True
print(isPalindrome("Able was I ere I saw Elba"))   # True
print(isPalindrome("Hello world"))                 # False
```

Bài 11: Giải thuật tính UCLN của hai số nguyên dương p và q (p > q) được mô tả như sau :

Gọi r là số dư trong phép chia q cho p:

+ Nếu r = 0 thì q là UCLN

+ Nếu r khác 0 thì gán p giá trị của q, gán cho q giá trị của r rồi lặp lại quá trình.

a) Xây dựng một định nghĩa đệ quy cho hàm UCLN(p, q)

b) Viết một giải thuật đệ quy và một giải thuật không đệ quy thể hiện hàm đó.

a) Định nghĩa đệ quy cho hàm UCLN(p, q)

Giải thuật Euclid tìm Ước chung lớn nhất của hai số nguyên dương p và q (p > q) được mô tả bằng định nghĩa đệ quy như sau: $UCLN(p,q) = \begin{cases} p & (q = 0) \\ UCLN(q, p \bmod q) & (q \neq 0) \end{cases}$

b) Viết một giải thuật đệ quy và một giải thuật không đệ quy thể hiện hàm đó.

Giải thuật đệ quy	Giải thuật không đệ quy

```python
def UCLN(p, q):
    if q == 0:
        return p
    else:
        return UCLN(q, p % q)
```

```python
def UCLN(p, q):
    while q != 0:
        r = p % q
        p = q
        q = r
    return p
```

*Cách bấm máy casio tìm UCLN(a, b) = ALPHA X (a SHIFT) b) =

*Ví dụ:

Gọi UCLN(48, 18):

48 % 18 = 12 → gọi UCLN(18, 12)

18 % 12 = 6 → gọi UCLN(12, 6)

12 % 6 = 0 → trả về 6

Kết quả: UCLN(48, 18) = 6

Bài 12: Cho một dãy số nguyên: $F(n) = \begin{cases} 1 & n = 1,2 \\ 2F(n-1) - 2F(n-2) & n > 2 \end{cases}$

a) Viết giải thuật đệ quy xác định giá trị của phần tử thứ n trong dãy.

b) Viết giải thuật không đệ quy xác định giá trị của phần tử thứ n trong dãy.

Giải thuật đệ quy	Giải thuật không đệ quy

```python
def F(n):
    if n == 1 or n == 2:
        return 1
    else:
        return 2 * F(n - 1) - 2 * F(n - 2)
```

```python
def F(n):
    if n == 1 or n == 2:
        return 1
    f1, f2 = 1, 1  # F(1) và F(2)
    for i in range(3, n + 1):
        fn = 2 * f2 - 2 * f1
        f1, f2 = f2, fn
    return fn
```

Bài 13: Thiết kế giải thuật tìm kiếm nhị phân theo kiểu đệ quy.

```python
def binarySearch_recursive(arr, left, right, x):
    if left > right:
        return -1  # Không tìm thấy
    mid = (left + right) // 2

    if arr[mid] == x:
        return mid
    elif arr[mid] > x:
        return binarySearch_recursive(arr, left, mid - 1, x)
    else:
```

```
        return binarySearch_recursive(arr, mid + 1, right, x)
arr = [1, 3, 5, 7, 9, 11, 13, 15]
x = 9
index = binarySearch_recursive(arr, 0, len(arr) - 1, x)
if index != -1:
    print(f"Phần tử {x} được tìm thấy tại chỉ số {index}")
else:
    print(f"Phần tử {x} không tồn tại trong danh sách")
```

Chương 3: Danh sách liên kết

Bài 1:

a. Cho một danh sách nối đơn có phần tử đầu tiên trỏ bởi L. Mỗi phần tử của danh sách gồm các thành phần sau:

- INFO: chứa thông tin của phần tử.
- LINK: là con trỏ, trỏ đến phần tử kế tiếp.

a_1) Lập hàm Count(L, S) thực hiện đếm số lượng các phần tử của danh sách thỏa mãn điều kiện INFO = S cho trước.

a_2) Lập thủ tục Move(L, k) thực hiện chuyển phần tử thứ k thành phần tử đầu tiên của danh sách.

b. Cho một danh sách nối kép trỏ bởi L và R (L trỏ đến phần tử đầu tiên, R trỏ đến phần tử cuối cùng của danh sách). Mỗi phần tử của danh sách gồm các thành phần sau:

- INFO: chứa thông tin của phần tử.
- LPTR và RPTR: là các con trỏ, trỏ đến phần tử liền trước và liền sau.

Lập thủ tục Delete(L, R, S) thực hiện loại bỏ phần tử đầu tiên thỏa mãn INFO = S ra khỏi danh sách.

a.

```
class Node:
    def __init__(self, info):
        self.INFO = info
        self.LINK = None
# Đếm số phần tử có INFO == S
def Count(L, S):
    count = 0
    current = L
    while current:
        if current.INFO == S:
            count += 1
        current = current.LINK
    return count
# Di chuyển phần tử thứ k lên đầu danh sách
def Move(L, k):
    if k <= 1 or not L or not L.LINK:
        return L
    prev = None
    curr = L
```

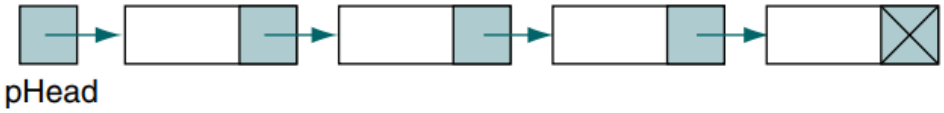
```
    i = 1
    while curr and i < k:
        prev = curr
        curr = curr.LINK
        i += 1
    if not curr:
        return L
    prev.LINK = curr.LINK
    curr.LINK = L
    return curr
# Duyệt và in danh sách liên kết đơn
def printList(L):
    print("Danh sách đơn:", end=" ")
    current = L
    while current:
        print(current.INFO, end=" -> ")
        current = current.LINK
    print("None")
def main():
    print("=== DANH SÁCH LIÊN KẾT ĐƠN ===")
    # Tạo danh sách đơn
    L = Node(5)
    L.LINK = Node(2)
    L.LINK.LINK = Node(3)
    L.LINK.LINK.LINK = Node(2)
    printList(L)
    print("Số phần tử có INFO = 2:", Count(L, 2))
    L = Move(L, 3)
    print("Sau khi Move phần tử thứ 3 lên đầu:")
    printList(L)
```

b.

```
class DoubleNode:
    def __init__(self, info):
        self.INFO = info
        self.LPTR = None  # Trỏ phần tử trước
        self.RPTR = None  # Trỏ phần tử sau
# Xoá phần tử đầu tiên có INFO == S
def Delete(L, R, S):
    current = L
    while current:
        if current.INFO == S:
            if current == L:
                L = current.RPTR
```

```
                if L:
                    L.LPTR = None
                else:
                    R = None
            elif current == R:
                R = current.LPTR
                if R:
                    R.RPTR = None
                else:
                    L = None
            else:
                current.LPTR.RPTR = current.RPTR
                current.RPTR.LPTR = current.LPTR
            break
        current = current.RPTR
    return L, R
# Duyệt và in danh sách liên kết kép
def printDoubleList(L):
    print("Danh sách kép:", end=" ")
    current = L
    while current:
        print(current.INFO, end=" <-> ")
        current = current.RPTR
    print("None")
def main():
    print("\n=== DANH SÁCH LIÊN KẾT KÉP ===")
    # Tạo danh sách kép: 7 <-> 4 <-> 9
    a = DoubleNode(7)
    b = DoubleNode(4)
    c = DoubleNode(9)
    a.RPTR = b
    b.LPTR = a
    b.RPTR = c
    c.LPTR = b
    L2 = a
    R2 = c
    printDoubleList(L2)
    L2, R2 = Delete(L2, R2, 4)
    print("Sau khi xóa phần tử có INFO = 4:")
    printDoubleList(L2)
if __name__ == "__main__":
    main()
```

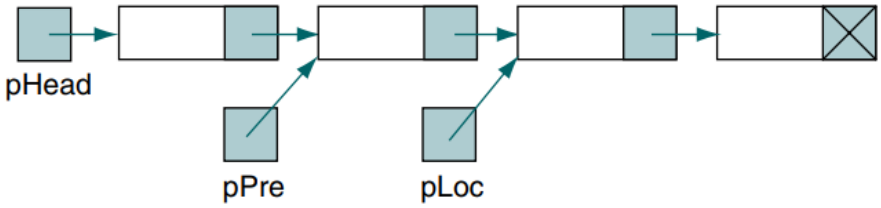
Bài 2: Bạn được yêu cầu xem xét điều gì xảy ra khi sử dụng câu lệnh sau trong một danh sách liên kết đơn như hình:



pHead = pHead → link

pHead thay đổi: ban đầu trỏ đến nút 1, bây giờ sẽ trỏ đến nút 2. Nút đầu tiên bị bỏ qua: không còn biến nào trỏ tới nút 1 nữa. Nếu không lưu nút đầu trước đó, bạn sẽ không thể truy cập lại nút 1 → nó bị mất khỏi danh sách. Danh sách giờ bắt đầu từ nút thứ 2.

Bài 3: Hãy tưởng tượng chúng ta có danh sách như trong Hình 5-26 được cài đặt dưới dạng một danh sách liên kết. Như đã thảo luận trong phần "Tìm kiếm trong danh sách" ở Mục 5.2, quá trình tìm kiếm cần phải có khả năng trả về cả vị trí của phần tử đứng trước (pPre) và vị trí của phần tử hiện tại (pLoc) dựa trên tiêu chí tìm kiếm.



Đoạn mã sau đây để gán giá trị cho pPre và pLoc chứa một lỗi phổ biến.

```
pLoc = pLoc->link
pPre = pPre->link
```

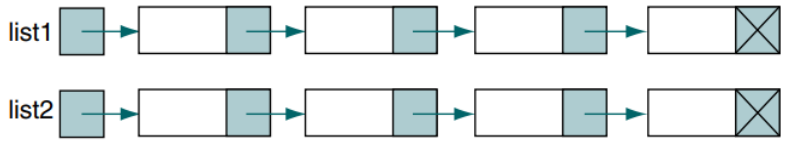
Lỗi đó là gì và làm thế nào để sửa?

(*Gợi ý: Nội dung ban đầu của các con trỏ này là gì khi bắt đầu tìm kiếm?*)

Lỗi hiện tại	**Cách sửa đúng**
pPre = pPre->link; *(Khi chạy đoạn code này, cả pLoc và pPre đều bị cập nhật cùng lúc → dẫn đến pPre không còn trỏ đến phần tử đứng ngay trước pLoc nữa.)*	pPre = pLoc;
Làm mất vị trí của phần tử trước	Giữ đúng quan hệ pPre - pLoc
Gây sai lệch khi cần xoá/sửa node	Dễ dẫn tới lỗi logic nghiêm trọng

Bài 4: Hãy tưởng tượng chúng ta có hai danh sách như trong hình bên dưới. Điều gì sẽ xảy ra nếu chúng ta áp dụng các câu lệnh sau cho hai danh sách đó?

```
pLoc = pLoc→link;
pPre = pPre→link;
list1 = list2;
```



1. pLoc = pLoc->link;
→ Di chuyển con trỏ pLoc sang nút kế tiếp trong danh sách mà nó đang duyệt.
Không ảnh hưởng đến list1 hay list2.
2. pPre = pPre->link;
→ Di chuyển con trỏ pPre sang nút kế tiếp.
Cũng không ảnh hưởng gì đến list1 và list2.
Hai lệnh đầu chỉ duyệt danh sách, không làm thay đổi cấu trúc.
3. list1 = list2;
Lệnh này gán địa chỉ đầu của list2 cho list1, tức là:
+) list1 không còn trỏ đến danh sách ban đầu của nó nữa.
+) list1 bây giờ trỏ đến cùng danh sách với list2.
Vậy kết quả cuối cùng là: list1 trỏ đến danh sách của list2
→ Sau lệnh gán, list1 và list2 cùng trỏ đến cùng một danh sách (là danh sách ban đầu của list2).

Bài 5: Điều gì sẽ xảy ra nếu chúng ta áp dụng các câu lệnh sau cho hai danh sách ở Bài 4?

```
1 set temp to list1
2 loop (temp link not null)
  1 set temp to temp link
3 end loop
4 set temp link to list2
```

1. set temp to list1
Khởi tạo con trỏ temp để trỏ tới đầu danh sách list1.
2. loop (temp link not null)
   1 set temp to temp link
3. end loop
Vòng lặp này sẽ:
Duyệt qua danh sách list1 bằng cách đi từng bước qua các liên kết link.
Dừng lại khi temp đang trỏ đến node cuối cùng (tức là temp->link == NULL).
4. set temp link to list2
Sau khi đến cuối list1, dòng này nối phần tử cuối cùng của list1 với đầu list2, tạo thành một danh sách liên kết mới: list1 → ... → cuối → list2 → ... → null
Kết quả:
Hai danh sách được nối lại với nhau:
list1 giờ bao gồm toàn bộ các node của list1 ban đầu nối tiếp với các node của list2.
list2 vẫn trỏ đến node đầu của danh sách 2, nhưng bây giờ cũng là một phần của danh sách list1.

Chương 4: Ngăn xếp (Stack)

Bài 1: Viết lại các thuật toán push, pop, stack.

```python
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class Stack:
```

```
    def __init__(self):
        self.top = None
    def is_empty(self):
        return self.top is None
    def push(self, data):
        new_node = Node(data)
        new_node.next = self.top
        self.top = new_node
        print(f"Đã push: {data}")
    def pop(self):
        if self.is_empty():
            print("Stack rỗng, không thể pop.")
            return None
        popped_data = self.top.data
        self.top = self.top.next
        print(f"Đã pop: {popped_data}")
        return popped_data
    def peek(self):
        if self.is_empty():
            print("Stack rỗng.")
            return None
        return self.top.data
    def display(self):
        current = self.top
        print("Stack hiện tại (trên cùng trước):")
        while current:
            print(current.data, end=" -> ")
            current = current.next
        print("None")
def main():
    s = Stack()
    s.push(10)
    s.push(20)
    s.push(30)
    s.display()
    print(f"Phần tử trên cùng (peek): {s.peek()}")
    s.pop()
    s.display()
    s.pop()
    s.pop()
    s.pop()
main()
```

Bài 2: Đảo số

```
class Node:
    def __init__(self, data):
        self.data = data
        self.link = None
class Stack:
    def __init__(self):
        self.top = None
        self.count = 0
    def pushStack(self, data):
        pNew = Node(data)
        pNew.link = self.top
        self.top = pNew
        self.count += 1
    def popStack(self):
        if self.top == None:
            print("danh sách rỗng")
        else:
            dataOut = self.top.data
            self.top = self.top.link
            self.count -= 1
        return dataOut
    def display(self):
        if self.top == None:
            print("Stack rong")
            return
        print("Stack(từ đỉnh xuống đáy)")
        current = self.top
        while current:
            print(f" {current.data}")
            current = current.link
    def dao_nguoc(self):
        if self.top == None:
            print("Stack rong")
            return
        temp_list = []
        while self.top != None:
            data = self.popStack()
            temp_list.append(data)

        print("\nIn theo thu tu dao nguoc:")
        for i in range(len(temp_list) -1, -1,-1):
            print(f" {temp_list[i]}")
```

```
def main():
    StackList = Stack()
    StackList.pushStack(4)
    StackList.pushStack(3)
    StackList.pushStack(2)
    StackList.pushStack(1)

    print("ngan xep ban dau")
    StackList.display()
    StackList.dao_nguoc()
main()
```

Bài 3: Chuyển hệ nhị phân

```
class _StackNode:
    def __init__(self, item):
        self.item = item
        self.next = None
class ngan_xep:
    def __init__(self):
        self.top = None
        self._size = 0
    def isEmpty(self):
        return self.top is None
    def length(self):
        return self._size
    def getTop(self):
        assert not self.isEmpty()
        return self.top.item
    def pop(self):
        assert not self.isEmpty()
        node = self.top
        self.top = self.top.next
        self._size -= 1
        return node.item
    def push(self, item):
        newNode = _StackNode(item)
        newNode.next = self.top
        self.top = newNode
        self._size += 1
def nhi_phan(n):
    if n == 0:
        return "0"
    S = ngan_xep()
    while n > 0:
```

```
        S.push(n % 2)
        n = n // 2
    binary = ""
    while not S.isEmpty():
        binary += str(S.pop())
    return binary
n = int(input("Nhập số nguyên hệ 10: "))
print("Số ở hệ nhị phân là:", nhi_phan(n))
```

Quy tắc chuyển trung tố sang hậu tố:

1. Toán hạng (biến/số) giữ nguyên thứ tự.
2. Toán tử đưa ra sau 2 toán hạng.
3. Dấu ngoặc dùng để xác định thứ tự ưu tiên, nhưng sẽ không xuất hiện trong hậu tố.
4. Thứ tự ưu tiên toán tử:
    - * và / > + và -
    - Phép toán có cùng độ ưu tiên thì tính từ trái sang phải

Quy tắc chuyển hậu tố sang trung tố:

1. Khởi tạo một stack rỗng.
2. Duyệt từng ký hiệu (toán hạng hoặc toán tử) trong biểu thức hậu tố:
    - Nếu là toán hạng (biến hoặc số):
      → Đẩy vào stack.
    - Nếu là toán tử (+, -, *, /...):
      → Lấy hai toán hạng trên cùng của stack ra (pop), gọi là op1 và op2.
      → Tạo chuỗi biểu thức trung tố: (op1 toán tử op2)
      → Đẩy kết quả này trở lại stack.
3. Sau khi duyệt hết, phần tử còn lại trong stack chính là biểu thức trung tố hoàn chỉnh.

Bài 4: Chuyển từ trung tố sang hậu tố.

a. D – B + C
b. A * B + C * D
c. (A + B) * C – D * F + C
d. (A – 2 * (B + C) – D * E) * F

a) DB – C +
b) A * B + C * D = ((A*B) + (C*D)) = ((AB*)(CD*)+) = AB*CD*+
c) (A+B) * C – D * F + C = ((((A+B)*C) – (D*F)) + C) = ((((AB+)C*)(DF*)-)C+) = AB+C*DF*-C+
d) (A – 2 * (B + C) – D * E) * F = ((((A – (2*(B+C))) – (D*E)) * F) = A2BC+*-DE*-F*

Bài 5: Chuyển từ hậu tố sang trung tố:

a. A B * C – D +
b. A B C + * D –

a) AB*C-D+ =

Bước 1: A là toán hạng → push A

Bước 2: B là toán hạng → push B → AB

Bước 3: * là toán tử → push(A*B)

Bước 4: C là toán hạng → push C → (A*B)C

Bước 5: - là toán tử → lấy hai phần tử (A*B) và C → push((A*B) – C)

Bước 6: D là toán hạng → push D → ((A*B) – C)D

Bước 7: + là toán hạng → lấy hai phần tử ((A*B) – C) và D → push (((A*B) – C) + D)

b) A B C + * D – =

Bước 1: A là toán hạng → push A

Bước 2: B là toán hạng → push B → A B

Bước 3: C là toán hạng → push C → A B C

Bước 4: + là toán tử → lấy hai phần tử B và C → push A(B+C)

Bước 5: * là toán tử → lấy hai phần tử A và (B+C) → push (A*(B+C))

Bước 6: D là toán hạng → push D → (A*(B+C))D

Bước 7: - là toán tử → lấy hai phần tử (A*(B+C)) và D → push ((A*(B+C)) – D)

Bài 6: Chuyển từ hậu tố sang trung tố, với các giá trị A, B, C và D lần lượt là 2, 3, 4 và 5.

a. A B * C – D +
b. A B C + * D –

a. Với các giá trị A = 2; B = 3; C = 4; D = 5, biểu thức trở thành: 2 3 * 4 – 5 +

Bước 1: Xét 2 là toán hạng → push 2

Bước 2: Xét 3 là toán hạng → push 3 → 2 3

Bước 3: Xét * là toán tử → lấy hai giá trị 2 và 3 → push (2*3) → push(6)

Bước 4: Xét 4 là toán hạng → push (6)4

Bước 5: Xét – là toán tử → lấy hai giá trị 6 và 4 → push(6 – 4) → push(2)

Bước 6: Xét 5 là toán hạng → push 5 → 2 5

Bước 7: Xét + là toán tử → lấy hai giá trị 2 và 5 → push(2 + 5) → push(7)

Kết quả là: 7

a. Với các giá trị A = 2; B = 3; C = 4; D = 5, biểu thức trở thành: 2 3 4 + * 5 –

Bước 1: Xét 2 là toán hạng → push 2

Bước 2: Xét 3 là toán hạng → push 3 → 2 3

Bước 3: Xét 4 là toán hạng → push 4 → 2 3 4

Bước 4: Xét + là toán tử → lấy hai phần tử 3 và 4 → push(3 + 4) → 2 7

Bước 5: Xét * là toán tử → lấy hai phần tử 2 và 7 → push (2 * 7) → 14

Bước 6: Xét 5 là toán hạng → push 5 → 14 5

Bước 7: Xét – là toán tử → lấy hai phần tử 14 và 5 → push 14 – 5 → push 9

Kết quả là: 9

Bài 7: Cho biểu thức:

$$A / (B - C) - D * E$$

a. Viết biểu thức hậu tố tương ứng với biểu thức đã cho.

b. Vẽ cây biểu diễn biểu thức đã cho. Viết kết quả của phép duyệt trước (pre-order) và phép duyệt sau (post-order) đối với cây thu được.

c. Minh họa quá trình định giá biểu thức hậu tố với: A = 8, B = 3, C = 1, D = 4, E = 2.

a.

B1: (B - C) → B C -

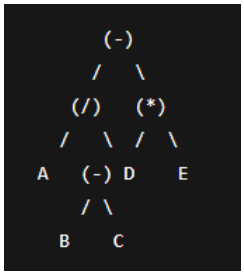
B2: A / (B - C) → A B C - /

B3: D * E → D E *

B4: (A / (B - C)) - (D * E) → A B C - / D E * -

Kết quả: A B C - / D E * -

b.

Cây	Duyệt trước Pre-order (root → left → right)	Duyệt sau Post-order (left → right → root):
	- / A - B C * D E	A B C - / D E * -

c.

Biểu thức: 8 3 1 - / 4 2 * -

3 1 - → 2

8 / 2 → 4

4 2 * → 8

4 - 8 → -4

Kết quả cuối cùng: -4

Chương 5: Hàng đợi (Queue)

Bài 1: Hãy tưởng tượng bạn có một ngăn xếp các số nguyên, S, và một hàng đợi các số nguyên, Q. Vẽ hình ảnh của S và Q sau các thao tác sau:

```
1  pushStack (S, 3)
2  pushStack (S, 12)
3  enqueue (Q, 5)
4  enqueue (Q, 8)
5  popStack (S, x)
6  pushStack (S, 2)
7  enqueue (Q, x)
8  dequeue (Q, y)
9  pushStack (S, x)
10 pushStack (S, y)
```

S = [ ] (ngăn xếp, đỉnh nằm bên phải)

Q = [ ] (hàng đợi, đầu hàng nằm bên trái)

Thực hiện từng thao tác:

1. pushStack(S, 3)

→ S = [3]

2. pushStack(S, 12)

→ S = [3, 12] (12 ở đỉnh)

3. enqueue(Q, 5)

→ Q = [5]

4. enqueue(Q, 8)
→ Q = [5, 8]
5. popStack(S, x)
→ Lấy 12 ra khỏi S, gán x = 12
→ S = [3]
6. pushStack(S, 2)
→ S = [3, 2]
7. enqueue(Q, x)
→ x = 12
→ Q = [5, 8, 12]
8. dequeue(Q, y)
→ Lấy phần tử đầu tiên trong Q: 5 → y = 5
→ Q = [8, 12]
9. pushStack(S, x)
→ x = 12
→ S = [3, 2, 12]
10. pushStack(S, y)
→ y = 5
→ S = [3, 2, 12, 5]
Stack S (từ đáy đến đỉnh): [3, 2, 12, 5] (Đỉnh là 5)
Queue Q (từ đầu đến cuối): [8, 12]

Bài 2: Giá trị của hàng đợi Q1, Q2 và ngăn xếp S sẽ là gì sau đoạn thuật toán sau:

1. x = 5 → ≠ 0 → push vào S = [5]
2. x = 6 → ≠ 0 → push vào S = [5, 6]
3. x = 9 → ≠ 0 → push vào S = [5, 6, 9]
4. x = 0 → bắt đầu cộng các phần tử trong S:
   - Pop 9 → z = 9
   - Pop 6 → z = 15
   - Pop 5 → z = 20

   → S = [ ] → enqueue(Q2, 20) → Q2 = [20]
5. x = 7 → push vào S = [7]
6. x = 5 → push vào S = [7, 5]
7. x = 0 → cộng stack:
   - Pop 5 → z = 5
   - Pop 7 → z = 12

   → S = [ ]

   → enqueue(Q2, 12) → Q2 = [20, 12]
8. x = 2 → push vào S = [2]
9. x = 6 → push vào S = [2, 6]

Kết quả cuối cùng:

- Q1: rỗng
- S: [2, 6] (đáy là 2, đỉnh là 6)
- Q2: [20, 12]

```
 1 S  = createStack
 2 Q1 = createQueue
 3 Q2 = createQueue
 4 enqueue (Q1, 5)
 5 enqueue (Q1, 6)
 6 enqueue (Q1, 9)
 7 enqueue (Q1, 0)
 8 enqueue (Q1, 7)
 9 enqueue (Q1, 5)
10 enqueue (Q1, 0)
11 enqueue (Q1, 2)
12 enqueue (Q1, 6)
13 loop (not emptyQueue (Q1))
   1 dequeue (Q1, x)
   2 if (x == 0)
     1 z = 0
     2 loop (not emptyStack (S))
       1 popStack (S, y)
       2 z = z + y
     3 end loop
     4 enqueue (Q2, z)
   3 else
     1 pushStack (S, x)
   4 end if
14 end loop
```

Bài 3: Nội dung của hàng đợi Q sẽ là gì sau khi đoạn mã sau được thực thi và các dữ liệu sau được nhập vào?
The data are: 5, 7, 12, 4, 0, 4, 6, 8, 67, 34, 23, 5, 0, 44, 33, 22, 6, 0.

```
1  Q = createQueue
2  loop (not end of file)
   1  read number
   2  if (number not 0)
      1  enqueue (Q, number)
   3  else
      1  x = queuerear (Q)
      2  enqueue (Q, x)
   4  end if
3  end loop
```

Khởi tạo hàng đợi Q.
Với mỗi số trong dữ liệu:
Nếu khác 0 → thêm số đó vào cuối hàng đợi.
Nếu bằng 0 → lấy phần tử cuối cùng trong hàng đợi (rear), rồi enqueue lại chính nó.

Bước	Số	Hành động	Q (sau bước đó)
1	5	enqueue(5)	[5]
2	7	enqueue(7)	[5, 7]
3	12	enqueue(12)	[5, 7, 12]
4	4	enqueue(4)	[5, 7, 12, 4]
5	0	queueRear = 4 → enqueue(4)	[5, 7, 12, 4, 4]
6	4	enqueue(4)	[5, 7, 12, 4, 4, 4]
7	6	enqueue(6)	[5, 7, 12, 4, 4, 4, 6]
8	8	enqueue(8)	[5, 7, 12, 4, 4, 4, 6, 8]
9	67	enqueue(67)	[5, 7, 12, 4, 4, 4, 6, 8, 67]
10	34	enqueue(34)	[5, 7, 12, 4, 4, 4, 6, 8, 67, 34]
11	23	enqueue(23)	[5, 7, 12, 4, 4, 4, 6, 8, 67, 34, 23]
12	5	enqueue(5)	[5, 7, 12, 4, 4, 4, 6, 8, 67, 34, 23, 5]
13	0	rear = 5 → enqueue(5)	[5, 7, 12, 4, 4, 4, 6, 8, 67, 34, 23, 5, 5]
14	44	enqueue(44)	[5, 7, 12, 4, 4, 4, 6, 8, 67, 34, 23, 5, 5, 44]
15	33	enqueue(33)	[5, 7, 12, 4, 4, 4, 6, 8, 67, 34, 23, 5, 5, 44, 33]
16	22	enqueue(22)	[5, 7, 12, 4, 4, 4, 6, 8, 67, 34, 23, 5, 5, 44, 33, 22]
17	6	enqueue(6)	[5, 7, 12, 4, 4, 4, 6, 8, 67, 34, 23, 5, 5, 44, 33, 22, 6]
18	0	rear = 6 → enqueue(6)	[5, 7, 12, 4, 4, 4, 6, 8, 67, 34, 23, 5, 5, 44, 33, 22, 6, 6]

Kết quả cuối cùng: [5, 7, 12, 4, 4, 4, 6, 8, 67, 34, 23, 5, 5, 44, 33, 22, 6, 6]

Bài 4: Nội dung của hàng đợi Q1 và hàng đợi Q2 sẽ là gì sau khi đoạn mã sau được thực thi và các dữ liệu sau được nhập vào? The data are 5, 7, 12, 4, 0, 4, 6.

```
1  Q1 = createQueue
2  Q2 = createQueue
3  loop (not end of file)
   1  read number
   2  enqueue (Q1, number)
   3  enqueue (Q2, number)
   4  loop (not empty Q1)
      1  dequeue (Q1, x)
      2  enqueue (Q2, x)
   5  end loop
4  end loop
```

*Giải thích thuật toán:

Mỗi lần đọc 1 số:

enqueue(Q1, number) → thêm vào Q1

enqueue(Q2, number) → thêm vào Q2

Sau đó: chuyển toàn bộ Q1 sang Q2 rồi Q1 bị xóa sạch

*Chạy thủ công:

Bước	Số đọc vào	Q1 trước khi chuyển	Q2 sau khi chuyển
1	5	[5]	[5, 5]
2	7	[7]	[5, 5, 7, 7]
3	12	[12]	[5, 5, 7, 7, 12, 12]
4	4	[4]	[5, 5, 7, 7, 12, 12, 4, 4]
5	0	[0]	[5, 5, 7, 7, 12, 12, 4, 4, 0, 0]
6	4	[4]	[5, 5, 7, 7, 12, 12, 4, 4, 0, 0, 4, 4]
7	6	[6]	[5, 5, 7, 7, 12, 12, 4, 4, 0, 0, 4, 4, 6, 6]

*Kết quả:

Hàng đợi	Giá trị cuối
Q1	[] (rỗng — luôn bị xóa sau mỗi vòng)
Q2	[5, 5, 7, 7, 12, 12, 4, 4, 0, 0, 4, 4, 6, 6]

Bài 5: What would be the contents of queue Q1 after the following code is executed and the following. The data are 5, 7, 12, 4, 0, 4, 6, 8, 67, 34, 23, 5, 0, 44, 33, 22, 6, 0.

```
1  Q1 = createQueue
2  S1 = createStack
3  loop (not end of file)
   1  read number
   2  if (number not 0)
      1  pushStack (S1, number)
   3  else
      1  popStack (S1, x)
      2  popStack (S1, x)
      3  loop (not empty S1)
         1  popStack (S1, x)
         2  enqueue (Q1, x)
      4  end loop
   4  end if
4  end loop
```

Khi đọc được một số ≠ 0, ta push vào stack S1

Khi gặp 0:

Pop 2 lần để bỏ 2 phần tử cuối cùng trong S1

Sau đó chuyển toàn bộ phần còn lại từ stack S1 sang hàng đợi Q1 (theo thứ tự ngược từ stack → queue)

Bước	Số đọc vào	Hành động	S1 (Stack sau bước)	Q1 (Queue sau bước)
1	5	push vào S1	[5]	[]
2	7	push vào S1	[5, 7]	[]
3	12	push vào S1	[5, 7, 12]	[]
4	4	push vào S1	[5, 7, 12, 4]	[]
5	0	pop 2 & chuyển còn lại sang Q1	[]	[7, 5]
6	4	push vào S1	[4]	[7, 5]
7	6	push vào S1	[4, 6]	[7, 5]
8	8	push vào S1	[4, 6, 8]	[7, 5]
9	67	push vào S1	[4, 6, 8, 67]	[7, 5]
10	34	push vào S1	[4, 6, 8, 67, 34]	[7, 5]
11	23	push vào S1	[4, 6, 8, 67, 34, 23]	[7, 5]
12	5	push vào S1	[4, 6, 8, 67, 34, 23, 5]	[7, 5]
13	0	pop 2 & chuyển còn lại sang Q1	[]	[7, 5, 34, 67, 8, 6, 4]
14	44	push vào S1	[44]	[7, 5, 34, 67, 8, 6, 4]
15	33	push vào S1	[44, 33]	[7, 5, 34, 67, 8, 6, 4]
16	22	push vào S1	[44, 33, 22]	[7, 5, 34, 67, 8, 6, 4]
17	6	push vào S1	[44, 33, 22, 6]	[7, 5, 34, 67, 8, 6, 4]
18	0	pop 2 & chuyển còn lại sang Q1	[]	[7, 5, 34, 67, 8, 6, 4, 33, 44]

Kết quả cuối cùng: Q1 = [7, 5, 34, 67, 8, 6, 4, 33, 44]; S1 = []

Bài 6: Hãy tưởng tượng rằng nội dung của hàng đợi Q1 và hàng đợi Q2 như được hiển thị. Nội dung của Q3 sẽ là gì sau khi đoạn mã sau được thực thi? Nội dung hàng đợi được hiển thị từ đầu (trái) đến cuối (phải).

Q1: 42 30 41 31 19 20 25 14 10 11 12 15

Q2: 4 5 4 10 13

```
1 Q3 = createQueue
2 count = 0
3 loop (not empty Q1 and not empty Q2)
  1 count = count + 1
  2 dequeue (Q1, x)
  3 dequeue (Q2, y)
  4 if (y equal count)
    1 enqueue (Q3, x)
  5 end if
1 end loop
```

Ta có:

Q1 ban đầu: 42 30 41 31 19 20 25 14 10 11 12 15

Q2 ban đầu: 4 5 4 10 13

Q3 rỗng

count = 0

Bước lặp	count	Q1 trước	Q2 trước	dequeue x	dequeue y	y == count?	Q3 sau bước này
1	1	42 30 41 31 19 20 25 14 10 11 12 15	4 5 4 10 13	42	4	4==1? N	(rỗng)
2	2	30 41 31 19 20 25 14 10 11 12 15	5 4 10 13	30	5	5==2? N	(rỗng)
3	3	41 31 19 20 25 14 10 11 12 15	4 10 13	41	4	4==3? N	(rỗng)
4	4	31 19 20 25 14 10 11 12 15	10 13	31	10	10==4? N	(rỗng)
5	5	19 20 25 14 10 11 12 15	13	19	13	13==5? N	(rỗng)

Sau bước 5, Q2 rỗng → thoát khỏi vòng lặp.

Kết luận cuối cùng: Q3 vẫn rỗng.

Bài 7: Chỉ sử dụng các thuật toán trong ADT hàng đợi (queue), viết một thuật toán ứng dụng có tên copyQueue để sao chép nội dung của một hàng đợi sang hàng đợi khác.

*Thuật toán:

```
copyQueue(Q1, Q2):
    n = size of Q1
    for i = 1 to n:
        x = dequeue(Q1)
        enqueue(Q1, x)   // đưa trả lại Q1 như ban đầu
        enqueue(Q2, x)   // thêm vào Q2
```

*Chương trình:

```
class Queue:
    def __init__(self):
```

```
        self.items = []
    def enqueue(self, x):
        self.items.append(x)
    def dequeue(self):
        if not self.isEmpty():
            return self.items.pop(0)
        else:
            return None
    def isEmpty(self):
        return len(self.items) == 0
    def front(self):
        if not self.isEmpty():
            return self.items[0]
        else:
            return None
    def size(self):
        return len(self.items)
    def __str__(self):
        return str(self.items)
def copyQueue(Q1, Q2):
    temp = Queue()
    while not Q1.isEmpty():
        x = Q1.dequeue()
        Q2.enqueue(x)      # copy sang Q2
        temp.enqueue(x)    # lưu tạm
    while not temp.isEmpty():
        x = temp.dequeue()
        Q1.enqueue(x)      # phục hồi Q1
if __name__ == "__main__":
    Q1 = Queue()
    for x in [10, 20, 30, 40]:
        Q1.enqueue(x)
    print("Q1 ban đầu:", Q1)
    Q2 = Queue()
    copyQueue(Q1, Q2)
    print("Q1 sau copy:", Q1)
    print("Q2 sao chép :", Q2)
```

Bài 8: Viết một thuật toán có tên stackToQueue để tạo một hàng đợi từ một ngăn xếp. Sau khi hàng đợi được tạo, đỉnh của ngăn xếp sẽ là đầu của hàng đợi và đáy của ngăn xếp sẽ là cuối của hàng đợi. Khi kết thúc thuật toán, ngăn xếp phải rỗng.

*Thuật toán:

stackToQueue(S, Q):

    while not S.isEmpty():

x = S.pop()

Q.enqueue(x)

*Chương trình:

```
class Stack:
    def __init__(self):
        self.items = []
    def push(self, x):
        self.items.append(x)
    def pop(self):
        return self.items.pop()
    def isEmpty(self):
        return len(self.items) == 0
    def __str__(self):
        return str(self.items)
class Queue:
    def __init__(self):
        self.items = []
    def enqueue(self, x):
        self.items.append(x)
    def dequeue(self):
        return self.items.pop(0)
    def isEmpty(self):
        return len(self.items) == 0
    def __str__(self):
        return str(self.items)
def stackToQueue(S, Q):
    while not S.isEmpty():
        x = S.pop()
        Q.enqueue(x)
if __name__ == "__main__":
    S = Stack()
    S.push(10)
    S.push(20)
    S.push(30)
    S.push(40)
    print("Stack ban đầu:", S)
    Q = Queue()
    stackToQueue(S, Q)
    print("Ngăn xếp sau khi chuyển:", S)
    print("Hàng đợi kết quả:", Q)
```

Bài 9: Viết một thuật toán có tên queueToStack để tạo một ngăn xếp từ một hàng đợi. Khi kết thúc thuật toán, hàng đợi phải không thay đổi; đầu của hàng đợi sẽ là đỉnh của ngăn xếp, và cuối của hàng đợi sẽ là đáy của ngăn xếp.

*Thuật toán:

queueToStack(Q, S):

    n = size(Q)

    for i = 1 to n:

        x = dequeue(Q)

        enqueue(Q, x)   // trả lại Q như cũ

        push(S, x)   // đẩy lên stack

*Chương trình:

```
class Queue:
    def __init__(self):
        self.items = []
    def enqueue(self, x):
        self.items.append(x)
    def dequeue(self):
        if not self.isEmpty():
            return self.items.pop(0)
        else:
            return None
    def isEmpty(self):
        return len(self.items) == 0
    def size(self):
        return len(self.items)
    def __str__(self):
        return str(self.items)
class Stack:
    def __init__(self):
        self.items = []
    def push(self, x):
        self.items.append(x)
    def pop(self):
        if not self.isEmpty():
            return self.items.pop()
        else:
            return None
    def isEmpty(self):
        return len(self.items) == 0
    def __str__(self):
        return str(self.items)

def queueToStack(Q, S):
```

```
    n = Q.size()
    for i in range(n):
        x = Q.dequeue()
        Q.enqueue(x)     # đưa trả lại Q như cũ
        S.push(x)        # đẩy vào stack
if __name__ == "__main__":
    Q = Queue()
    for x in [10, 20, 30, 40]:
        Q.enqueue(x)
    print("Hàng đợi ban đầu:", Q)
    S = Stack()
    queueToStack(Q, S)
    print("Hàng đợi sau khi chuyển:", Q)
    print("Ngăn xếp kết quả:", S)
```

Bài 10: Viết một thuật toán nén một chuỗi bằng cách xóa tất cả các ký tự khoảng trắng trong chuỗi. Một cách để làm điều này là sử dụng một hàng đợi các ký tự. Chèn các ký tự không phải khoảng trắng từ chuỗi vào hàng đợi. Khi bạn đến cuối chuỗi, lấy các ký tự ra khỏi hàng đợi và đặt chúng trở lại vào chuỗi.

*Thuật toán:

compressString(s):

Q = empty queue

for i = 0 to len(s)-1:

if s[i] != ' ':

Q.enqueue(s[i])

result = ""

while not Q.isEmpty():

c = Q.dequeue()

result = result + c

return result

*Chương trình:

```
class Queue:
    def __init__(self):
        self.items = []
    def enqueue(self, x):
        self.items.append(x)
    def dequeue(self):
        if not self.isEmpty():
            return self.items.pop(0)
        else:
            return None
    def isEmpty(self):
        return len(self.items) == 0
def compressString(s):
    Q = Queue()
```

```
    for c in s:
        if c != ' ':
            Q.enqueue(c)
    result = ""
    while not Q.isEmpty():
        result += Q.dequeue()
    return result
if __name__ == "__main__":
    s = "  a b  c d e  "
    print("Chuỗi ban đầu: [" + s + "]")
    compressed = compressString(s)
    print("Chuỗi sau nén: [" + compressed + "]")
```

Bài 11: Cho một hàng đợi các số nguyên, viết một thuật toán mà, chỉ sử dụng ADT hàng đợi, tính toán và in ra tổng và trung bình của các số nguyên trong hàng đợi mà không làm thay đổi nội dung của hàng đợi.

*Thuật toán:

sumAverageQueue(Q):

  sum = 0

  count = 0

  n = size(Q)

  for i = 1 to n:

    x = dequeue(Q)

    sum = sum + x

    count = count + 1

    enqueue(Q, x) // trả lại Q như ban đầu

  avg = sum / count

  print sum, avg

*Chương trình:

```
class Queue:
    def __init__(self):
        self.items = []
    def enqueue(self, x):
        self.items.append(x)
    def dequeue(self):
        if not self.isEmpty():
            return self.items.pop(0)
        else:
            return None
    def isEmpty(self):
        return len(self.items) == 0
    def size(self):
        return len(self.items)
    def __str__(self):
        return str(self.items)
```

```
def sumAverageQueue(Q):
    total = 0
    count = 0
    n = Q.size()
    for i in range(n):
        x = Q.dequeue()
        total += x
        count += 1
        Q.enqueue(x)     # trả lại Q như cũ
    average = total / count if count > 0 else 0
    print("Tổng =", total)
    print("Trung bình =", average)
if __name__ == "__main__":
    Q = Queue()
    for x in [5, 10, 15, 20]:
        Q.enqueue(x)
    print("Queue ban đầu:", Q)
    sumAverageQueue(Q)
    print("Queue sau tính toán:", Q)
```

Bài 12: Cho một hàng đợi các số nguyên, viết một thuật toán xóa tất cả các số nguyên âm mà không làm thay đổi thứ tự của các phần tử còn lại trong hàng đợi.

*Thuật toán:

removeNegative(Q):

n = size(Q)

for i = 1 to n:

x = dequeue(Q)

if x >= 0:

enqueue(Q, x)

// kết thúc, Q chỉ còn số không âm, thứ tự vẫn giữ nguyên

*Chương trình:

```
class Queue:
    def __init__(self):
        self.items = []
    def enqueue(self, x):
        self.items.append(x)
    def dequeue(self):
        if not self.isEmpty():
            return self.items.pop(0)
        else:
            return None
    def isEmpty(self):
        return len(self.items) == 0
    def size(self):
```

```
        return len(self.items)
    def __str__(self):
        return str(self.items)
def removeNegative(Q):
    n = Q.size()
    for i in range(n):
        x = Q.dequeue()
        if x >= 0:
            Q.enqueue(x)
if __name__ == "__main__":
    Q = Queue()
    for x in [3, -5, 8, -2, 7, -9, 4]:
        Q.enqueue(x)
    print("Queue ban đầu:", Q)
    removeNegative(Q)
    print("Queue sau khi xóa số âm:", Q)
```

Bài 13: Viết một thuật toán đảo ngược nội dung của một hàng đợi.

*Thuật toán:

reverseQueue(Q):
    S = empty stack
    while not Q.isEmpty():
        x = Q.dequeue()
        S.push(x)
    while not S.isEmpty():
        x = S.pop()
        Q.enqueue(x)

*Chương trình:

```
class Queue:
    def __init__(self):
        self.items = []
    def enqueue(self, x):
        self.items.append(x)
    def dequeue(self):
        if not self.isEmpty():
            return self.items.pop(0)
        else:
            return None
    def isEmpty(self):
        return len(self.items) == 0
    def __str__(self):
        return str(self.items)
class Stack:
    def __init__(self):
```

```
        self.items = []
    def push(self, x):
        self.items.append(x)
    def pop(self):
        if not self.isEmpty():
            return self.items.pop()
        else:
            return None
    def isEmpty(self):
        return len(self.items) == 0
def reverseQueue(Q):
    S = Stack()
    while not Q.isEmpty():
        x = Q.dequeue()
        S.push(x)
    while not S.isEmpty():
        x = S.pop()
        Q.enqueue(x)
if __name__ == "__main__":
    Q = Queue()
    for x in [1, 2, 3, 4, 5]:
        Q.enqueue(x)
    print("Queue ban đầu:", Q)
    reverseQueue(Q)
    print("Queue sau đảo ngược:", Q)
```

Bài 14: Sử dụng kiểu dữ liệu trừu tượng danh sách liên kết được mô tả trong Phần 4.3 "ADT Hàng đợi", viết một giao diện người dùng dựa trên menu để kiểm tra từng thao tác trong ADT. Bất kỳ lỗi nào được phát hiện trong quá trình xử lý nên được in ra như một phần của kết quả kiểm tra. Một menu gợi ý được hiển thị bên dưới:

```
A. Enqueue data into queue
B. Dequeue and print data
C. Print data at the front
D. Print data at the rear
E. Print entire queue
F. Print queue status: Empty
G. Print queue status: Full
H. Print number of elements and quit
```

Kiểm tra chương trình của bạn với các trường hợp kiểm thử sau. Bạn có thể bao gồm các trường hợp kiểm thử bổ sung sau khi đã thực hiện các kiểm thử được hiển thị bên dưới.

a. In trạng thái hàng đợi, Rỗng (F).

b. Xóa và in dữ liệu (B). Nên trả về lỗi.

c. Thêm dữ liệu vào hàng đợi: 5 (A).

d. Thêm dữ liệu vào hàng đợi: 8 (A).

e. In trạng thái hàng đợi, Rỗng (F).

f. In trạng thái hàng đợi, Đầy (G).
g. In dữ liệu ở đầu (C).
h. In dữ liệu ở cuối (D).
i. In toàn bộ hàng đợi (E).
j. In số lượng phần tử trong hàng đợi (H).
k. Xóa và in dữ liệu (B).
l. Xóa và in dữ liệu (B).
m. Xóa và in dữ liệu (B). Nên trả về lỗi.
n. Thêm dữ liệu vào hàng đợi: 14 (A).
o. In dữ liệu ở đầu (C).
p. In dữ liệu ở cuối (D).
q. Thêm dữ liệu vào hàng đợi: 32 (A).
r. In dữ liệu ở đầu (C).
s. In dữ liệu ở cuối (D).
t. Hủy hàng đợi và thoát (I).

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
class LinkedQueue:
    def __init__(self, capacity=None):
        self.front_node = None
        self.rear_node = None
        self.count = 0
        self.capacity = capacity  # None nghĩa là không giới hạn
    def isEmpty(self):
        return self.count == 0
    def isFull(self):
        if self.capacity is None:
            return False
        return self.count >= self.capacity
    def enqueue(self, data):
        if self.isFull():
            print("ERROR: Queue is full.")
            return
        newNode = Node(data)
        if self.isEmpty():
            self.front_node = newNode
            self.rear_node = newNode
        else:
            self.rear_node.next = newNode
            self.rear_node = newNode
        self.count += 1
```

```
    def dequeue(self):
        if self.isEmpty():
            print("ERROR: Queue is empty, cannot dequeue.")
            return None
        value = self.front_node.data
        self.front_node = self.front_node.next
        self.count -= 1
        if self.front_node is None:
            self.rear_node = None
        return value
    def front(self):
        if self.isEmpty():
            print("ERROR: Queue is empty.")
            return None
        return self.front_node.data
    def rear(self):
        if self.isEmpty():
            print("ERROR: Queue is empty.")
            return None
        return self.rear_node.data
    def size(self):
        return self.count
    def printQueue(self):
        temp = self.front_node
        result = []
        while temp is not None:
            result.append(temp.data)
            temp = temp.next
        print("Queue contents:", result)
    def clear(self):
        self.front_node = None
        self.rear_node = None
        self.count = 0
        print("Queue cleared.")
def menu():
    print("""
Menu:
A. Enqueue data into queue
B. Dequeue and print data
C. Print data at the front
D. Print data at the rear
E. Print entire queue
F. Print queue status: Empty
```

```
G. Print queue status: Full
H. Print number of elements
I. Clear queue and quit
""")
def test():
    Q = LinkedQueue()  # không đặt capacity, linked list nên không đầy
    while True:
        menu()
        choice = input("Choose an option: ").upper()
        if choice == "A":
            data = input("Enter data to enqueue: ")
            Q.enqueue(data)
        elif choice == "B":
            data = Q.dequeue()
            if data is not None:
                print("Dequeued:", data)
        elif choice == "C":
            data = Q.front()
            if data is not None:
                print("Front data:", data)
        elif choice == "D":
            data = Q.rear()
            if data is not None:
                print("Rear data:", data)
        elif choice == "E":
            Q.printQueue()
        elif choice == "F":
            if Q.isEmpty():
                print("Queue is empty.")
            else:
                print("Queue is not empty.")
        elif choice == "G":
            if Q.isFull():
                print("Queue is full.")
            else:
                print("Queue is not full (unlimited linked list).")
        elif choice == "H":
            print("Number of elements:", Q.size())
        elif choice == "I":
            Q.clear()
            print("Goodbye!")
            break
        else:
```

```
            print("Invalid option. Please try again.")
if __name__ == "__main__":
    test()
```

Bài 15: Sử dụng ADT C (kiểu dữ liệu trừu tượng C), viết một chương trình mô phỏng hoạt động của một hệ thống điện thoại có thể tìm thấy trong một doanh nghiệp nhỏ, chẳng hạn như cửa hàng pizza địa phương của bạn. Chỉ một người có thể trả lời điện thoại (một hàng đợi máy chủ đơn), nhưng có thể có số lượng cuộc gọi chờ trả lời không giới hạn.

Phân tích hàng đợi xem xét hai yếu tố chính: thời gian người yêu cầu chờ đợi dịch vụ (thời gian chờ trong hàng đợi - trong trường hợp này, khách hàng gọi đặt pizza) và thời gian phục vụ (thời gian cần thiết để khách hàng đặt hàng). Chương trình của bạn nên mô phỏng hoạt động của điện thoại và thu thập số liệu thống kê trong quá trình.

Chương trình yêu cầu hai đầu vào để chạy mô phỏng: (1) thời gian hoạt động dịch vụ tính bằng giờ và (2) thời gian tối đa để tổng đài viên nhận một đơn hàng (thời gian phục vụ tối đa).

Bốn yếu tố được yêu cầu để chạy mô phỏng: một vòng lặp thời gian, một trình mô phỏng cuộc gọi, một bộ xử lý cuộc gọi và một hàm bắt đầu cuộc gọi.

1. Vòng lặp thời gian: Đây đơn giản là vòng lặp mô phỏng. Mỗi lần lặp của vòng lặp được coi là 1 phút trong thời gian thực. Vòng lặp tiếp tục cho đến khi dịch vụ đã hoạt động đủ thời gian yêu cầu (xem đầu vào ở trên). Tuy nhiên, khi thời gian hoạt động hoàn tất, bất kỳ cuộc gọi nào đang chờ phải được trả lời trước khi kết thúc mô phỏng. Vòng lặp thời gian có các chức năng phụ sau:

- Xác định xem một cuộc gọi có được nhận hay không (trình mô phỏng cuộc gọi)
- Xử lý cuộc gọi đang hoạt động
- Bắt đầu cuộc gọi mới Trình tự này cho phép một cuộc gọi được hoàn thành và một cuộc gọi khác được bắt đầu trong cùng một phút.

2. Trình mô phỏng cuộc gọi: Trình mô phỏng cuộc gọi sử dụng một bộ tạo số ngẫu nhiên để xác định xem một cuộc gọi có được nhận hay không. Chia tỷ lệ số ngẫu nhiên thành một phạm vi thích hợp, ví dụ từ 1 đến 10. Số ngẫu nhiên nên được so sánh với một hằng số đã định nghĩa. Nếu giá trị nhỏ hơn hằng số, một cuộc gọi đã được nhận; nếu không, không có cuộc gọi nào được nhận. Để mô phỏng, hãy đặt mức gọi là 50%; nghĩa là, trung bình, một cuộc gọi được nhận cứ sau 2 phút. Nếu một cuộc gọi được nhận, hãy đặt nó vào một hàng đợi.

3. Xử lý cuộc gọi đang hoạt động: Nếu một cuộc gọi đang hoạt động, hãy kiểm tra xem nó đã hoàn thành hay chưa. Nếu đã hoàn thành, in ra số liệu thống kê cho cuộc gọi hiện tại và thu thập các số liệu thống kê cần thiết cho báo cáo cuối công việc.

4. Bắt đầu cuộc gọi mới: Nếu không có cuộc gọi nào đang hoạt động, hãy bắt đầu một cuộc gọi mới nếu có một cuộc gọi đang chờ trong hàng đợi. Lưu ý rằng việc bắt đầu một cuộc gọi phải tính toán thời gian cuộc gọi đã chờ.

Trong quá trình xử lý, hãy in dữ liệu được hiển thị trong Bảng 4-2 sau khi mỗi cuộc gọi hoàn thành. (Lưu ý: bạn sẽ không nhận được cùng một kết quả.)"

Clock time	Call number	Arrival time	Wait time	Start time	Service time	Queue size
4	1	2	0	2	3	2
6	2	3	2	5	2	4

Khi kết thúc mô phỏng, hãy in ra các số liệu thống kê sau đã thu thập được trong quá trình xử lý. Đảm bảo sử dụng định dạng thích hợp cho mỗi số liệu thống kê, chẳng hạn như số thập phân cho giá trị trung bình.

a. Tổng số cuộc gọi: các cuộc gọi nhận được trong thời gian hoạt động.
b. Tổng thời gian nhàn rỗi: tổng thời gian không có cuộc gọi nào được phục vụ.
c. Tổng thời gian chờ: tổng thời gian chờ của tất cả các cuộc gọi.
d. Tổng thời gian phục vụ: tổng thời gian phục vụ của tất cả các cuộc gọi.
e. Kích thước hàng đợi tối đa: số lượng cuộc gọi chờ lớn nhất trong quá trình mô phỏng.
f. Thời gian chờ trung bình: tổng thời gian chờ / số lượng cuộc gọi.
g. Thời gian phục vụ trung bình: tổng thời gian phục vụ / số lượng cuộc gọi.
Chạy trình mô phỏng hai lần. Cả hai lần chạy đều mô phỏng trong 2 giờ. Trong lần mô phỏng đầu tiên, sử dụng thời gian phục vụ tối đa là 2 phút. Trong lần chạy thứ hai, sử dụng thời gian phục vụ tối đa là 5 phút.

```
class Queue:
    def __init__(self):
        self.data = []
    def enqueue(self, x):
        self.data.append(x)
    def dequeue(self):
        if not self.is_empty():
            return self.data.pop(0)
        return None
    def is_empty(self):
        return len(self.data) == 0
    def size(self):
        return len(self.data)
class RandomGen:
    def __init__(self, seed=12345):
        self.state = seed
    def next(self):
        self.state = (1103515245 * self.state + 12345) % (2**31)
        return self.state % 10 + 1    # giá trị từ 1..10
def simulate(service_hours, max_service_time):
    total_time = service_hours * 60
    queue = Queue()
    rng = RandomGen()
    clock = 0
    call_number = 0
    current_call = None
    current_call_remaining = 0
    total_calls = 0
    total_idle_time = 0
    total_wait_time = 0
    total_service_time = 0
    max_queue_size = 0
    print("Clock | Call# | Arrival | Wait | Start | Service | QueueSize")
    print("-----------------------------------------------------------")
```

```
while clock < total_time or not queue.is_empty() or current_call:
    # 1. Xác định xem có cuộc gọi mới hay không
    r = rng.next()
    if clock < total_time and r <= 5:  # 50% nhận cuộc gọi
        call_number += 1
        call = {
            'number': call_number,
            'arrival_time': clock,
        }
        queue.enqueue(call)
        total_calls += 1
    # 2. Xử lý cuộc gọi đang hoạt động
    if current_call:
        current_call_remaining -= 1
        if current_call_remaining == 0:
            # kết thúc cuộc gọi
            print(f"{clock:5d} | {current_call['number']:5d} |"
                  f"{current_call['arrival_time']:7d} |"
                  f"{current_call['wait_time']:4d} |"
                  f"{current_call['start_time']:5d} |"
                  f"{current_call['service_time']:7d} |"
                  f"{queue.size():9d}")
            total_wait_time += current_call['wait_time']
            total_service_time += current_call['service_time']
            current_call = None
    else:
        total_idle_time += 1
    # 3. Bắt đầu cuộc gọi mới nếu có
    if not current_call and not queue.is_empty():
        current_call = queue.dequeue()
        current_call['wait_time'] = clock - current_call['arrival_time']
        current_call['start_time'] = clock
        current_call['service_time'] = rng.next() % max_service_time + 1
        current_call_remaining = current_call['service_time']
    if queue.size() > max_queue_size:
        max_queue_size = queue.size()
    clock += 1
avg_wait = total_wait_time / total_calls if total_calls else 0
avg_service = total_service_time / total_calls if total_calls else 0
print("\n*** Kết quả mô phỏng ***")
print(f"Tổng số cuộc gọi: {total_calls}")
print(f"Tổng thời gian nhàn rỗi: {total_idle_time} phút")
print(f"Tổng thời gian chờ: {total_wait_time} phút")
```

```
    print(f"Tổng thời gian phục vụ: {total_service_time} phút")
    print(f"Kích thước hàng đợi lớn nhất: {max_queue_size}")
    print(f"Thời gian chờ trung bình: {avg_wait:.2f} phút")
    print(f"Thời gian phục vụ trung bình: {avg_service:.2f} phút")
print("=== MÔ PHỎNG LẦN 1 (service max 2 phút) ===")
simulate(2, 2)
print("\n=== MÔ PHỎNG LẦN 2 (service max 5 phút) ===")
simulate(2, 5)
```

Bài 16: Lặp lại mô phỏng hàng đợi trong Bài tập 22, sử dụng nhiều máy chủ hàng đợi như bạn có thể tìm thấy trong một ngân hàng. Chỉ nên có một hàng đợi duy nhất. Số lượng máy chủ, được đọc từ bàn phím, có thể dao động từ 2 đến 5.

Để mô phỏng nhiều máy chủ, bạn cần cung cấp một cấu trúc trạng thái khách hàng riêng cho mỗi máy chủ. Trong vòng lặp xử lý, mỗi máy chủ nên được kiểm tra xem đã hoàn thành hay chưa. Tương tự, trong chức năng "bắt đầu cuộc gọi mới", một cuộc gọi nên được bắt đầu cho mỗi máy chủ rảnh rỗi.

Bạn cũng cần sửa đổi trình mô phỏng cuộc gọi để xử lý tối đa 3 khách hàng đến cùng một lúc. Để xác định số lượng khách hàng, hãy tạo một số ngẫu nhiên trong phạm vi từ 0 đến 3. Nếu số đó là 0, không có khách hàng nào đến. Nếu số đó không phải là 0, hãy thêm số lượng khách hàng được chỉ định bởi số ngẫu nhiên vào hàng đợi.

```
class Queue:
    def __init__(self):
        self.data = []
    def enqueue(self, x):
        self.data.append(x)
    def dequeue(self):
        if not self.is_empty():
            return self.data.pop(0)
        return None
    def is_empty(self):
        return len(self.data) == 0
    def size(self):
        return len(self.data)
class RandomGen:
    def __init__(self, seed=12345):
        self.state = seed
    def next(self):
        self.state = (1103515245 * self.state + 12345) % (2**31)
        return self.state % 10 + 1  # giá trị 1..10
    def next_custom(self, maxval):
        # trả giá trị 0..maxval
        self.state = (1103515245 * self.state + 12345) % (2**31)
        return self.state % (maxval+1)
def simulate_multi(service_hours, max_service_time, num_servers):
    total_time = service_hours * 60
    queue = Queue()
```

```
    rng = RandomGen()
    servers = []
    for i in range(num_servers):
        servers.append({
            'busy': False,
            'current_call': None,
            'remaining_time': 0
        })
    clock = 0
    call_number = 0
    total_calls = 0
    total_idle_time = [0]*num_servers
    total_wait_time = 0
    total_service_time = 0
    max_queue_size = 0

    print("Clock | Server | Call# | Arrival | Wait | Start | Service | QueueSize")
    print("-----------------------------------------------------------")
    while clock < total_time or not queue.is_empty() or any(s['busy'] for s in servers):
        r = rng.next_custom(3)
        if clock < total_time and r > 0:
            for _ in range(r):
                call_number += 1
                call = {
                    'number': call_number,
                    'arrival_time': clock,
                }
                queue.enqueue(call)
                total_calls += 1
        if queue.size() > max_queue_size:
            max_queue_size = queue.size()
        for idx in range(num_servers):
            server = servers[idx]
            if server['busy']:
                server['remaining_time'] -= 1
                if server['remaining_time'] == 0:
                    c = server['current_call']
                    print(f"{clock:5d} | {idx+1:6d} | {c['number']:5d} |"
                          f"{c['arrival_time']:7d} |"
                          f"{c['wait_time']:4d} |"
                          f"{c['start_time']:5d} |"
                          f"{c['service_time']:7d} |"
                          f"{queue.size():9d}")
```

```
                    total_wait_time += c['wait_time']
                    total_service_time += c['service_time']
                    server['busy'] = False
                    server['current_call'] = None
            else:
                total_idle_time[idx] += 1
            if not server['busy'] and not queue.is_empty():
                c = queue.dequeue()
                c['wait_time'] = clock - c['arrival_time']
                c['start_time'] = clock
                c['service_time'] = rng.next() % max_service_time + 1
                server['busy'] = True
                server['current_call'] = c
                server['remaining_time'] = c['service_time']
        clock += 1
    avg_wait = total_wait_time / total_calls if total_calls else 0
    avg_service = total_service_time / total_calls if total_calls else 0
    total_idle = sum(total_idle_time)
    print("\n*** Kết quả mô phỏng ***")
    print(f"Tổng số cuộc gọi: {total_calls}")
    print(f"Tổng thời gian nhàn rỗi: {total_idle} phút")
    print(f"Tổng thời gian chờ: {total_wait_time} phút")
    print(f"Tổng thời gian phục vụ: {total_service_time} phút")
    print(f"Kích thước hàng đợi lớn nhất: {max_queue_size}")
    print(f"Thời gian chờ trung bình: {avg_wait:.2f} phút")
    print(f"Thời gian phục vụ trung bình: {avg_service:.2f} phút")
while True:
    n = int(input("Nhập số máy chủ (2-5): "))
    if 2 <= n <= 5:
        break
print("\n=== MÔ PHỎNG NGÂN HÀNG NHIỀU SERVER ===")
simulate_multi(2, 5, n)
```

Bài 17: Hàng đợi thường được sử dụng trong các hệ thống mạng. Ví dụ, email được đặt vào hàng đợi khi chờ được gửi đi và sau khi nó đến hộp thư của người nhận. Tuy nhiên, một vấn đề xảy ra nếu bộ xử lý thư đi không thể gửi một hoặc nhiều tin nhắn trong hàng đợi. Chẳng hạn, một tin nhắn có thể không được gửi đi vì hệ thống của người nhận không khả dụng.

Viết một trình mô phỏng email xử lý thư với tốc độ trung bình 40 tin nhắn mỗi phút. Khi các tin nhắn được nhận, chúng được đặt vào một hàng đợi. Đối với mô phỏng, giả sử rằng các tin nhắn đến với tốc độ trung bình 30 tin nhắn mỗi phút. Hãy nhớ rằng, các tin nhắn phải đến ngẫu nhiên, vì vậy bạn cần sử dụng bộ tạo số ngẫu nhiên để xác định khi nào tin nhắn được nhận.

Mỗi phút, bạn có thể lấy ra khỏi hàng đợi (dequeue) tối đa 40 tin nhắn và gửi chúng. Giả sử rằng 25% các tin nhắn trong hàng đợi không thể được gửi trong bất kỳ chu kỳ xử lý nào. Một lần nữa, bạn cần sử dụng một số ngẫu nhiên để xác định xem một tin nhắn nhất định có thể được gửi hay không. Nếu không thể gửi, hãy đặt nó

trở lại cuối hàng đợi (enqueue lại).
Chạy mô phỏng trong 24 giờ, theo dõi số lần mỗi tin nhắn phải được xếp hàng đợi lại. Khi kết thúc mô phỏng, in ra các số liệu thống kê hiển thị:
a. Tổng số tin nhắn đã xử lý
b. Tốc độ đến trung bình
c. Số lượng tin nhắn trung bình được gửi mỗi phút
d. Số lượng tin nhắn trung bình trong hàng đợi mỗi phút e. Số lượng tin nhắn được gửi ở lần thử đầu tiên, số lượng được gửi ở lần thử thứ hai, v.v.
f. Số lần trung bình tin nhắn phải được đưa vào hàng đợi.

```
class Queue:
    def __init__(self):
        self.data = []
    def enqueue(self, x):
        self.data.append(x)
    def dequeue(self):
        if not self.is_empty():
            return self.data.pop(0)
        return None
    def is_empty(self):
        return len(self.data) == 0
    def size(self):
        return len(self.data)
class RandomGen:
    def __init__(self, seed=98765):
        self.state = seed
    def next(self):
        self.state = (1103515245 * self.state + 12345) % (2**31)
        return self.state
    def rand(self):
        return self.next() % 100 + 1  # trả 1..100
def simulate_email():
    queue = Queue()
    rng = RandomGen()
    total_minutes = 24 * 60  # 1440 phút
    total_processed = 0
    total_arrivals = 0
    total_sent = 0
    total_queue_size = 0
    retry_count_map = {}  # key = số lần thử, value = số tin
    class Email:
        def __init__(self, arrival_time):
            self.retries = 0
            self.arrival_time = arrival_time
```

```
    for minute in range(total_minutes):
        arrivals = 0
        for i in range(60):
            if rng.rand() <= 50:  # 50% mỗi giây => trung bình ~30/phút
                arrivals += 1
        for _ in range(arrivals):
            queue.enqueue(Email(minute))
        total_arrivals += arrivals
        sent_this_minute = 0
        count = 0
        while not queue.is_empty() and count < 40:
            email = queue.dequeue()
            count += 1
            total_processed += 1
            if rng.rand() <= 25:
                email.retries += 1
                queue.enqueue(email)
            else:
                sent_this_minute += 1
                times = email.retries + 1
                retry_count_map[times] = retry_count_map.get(times, 0) + 1
        total_sent += sent_this_minute
        total_queue_size += queue.size()
    avg_arrival_rate = total_arrivals / total_minutes
    avg_sent_per_minute = total_sent / total_minutes
    avg_queue_size = total_queue_size / total_minutes
    avg_retry_per_email = 0
    total_emails_sent = sum(retry_count_map.values())
    if total_emails_sent > 0:
        sum_retry = sum((k-1)*v for k,v in retry_count_map.items())
        avg_retry_per_email = sum_retry / total_emails_sent
    print("\n=== KẾT QUẢ MÔ PHỎNG EMAIL ===")
    print(f"Tổng tin đã xử lý: {total_processed}")
    print(f"Tốc độ đến trung bình: {avg_arrival_rate:.2f} tin/phút")
    print(f"Số lượng tin gửi trung bình mỗi phút: {avg_sent_per_minute:.2f}")
    print(f"Số lượng tin trung bình trong queue mỗi phút: {avg_queue_size:.2f}")
    print("\nSố tin gửi thành công theo số lần thử:")
    for retry, num in sorted(retry_count_map.items()):
        print(f"  Gửi thành công lần thử thứ {retry}: {num} tin")
    print(f"Số lần thử trung bình trên mỗi tin nhắn: {avg_retry_per_email:.2f}")
simulate_email()
```

Bài 18: Cho hàng đợi Q gồm các giá trị [i, j, m, n], trong đó:

- Đầu hàng đợi là i (bên trái) – nghĩa là phần tử này sẽ bị lấy ra đầu tiên.
- Đuôi hàng đợi là n (bên phải) – nghĩa là phần tử này được thêm vào sau cùng.

i, j, m, n là những giá trị nguyên nào đó (không quan trọng cụ thể là bao nhiêu).

Bạn có một ngăn xếp S rỗng.

a) Cho biết tình trạng của hàng đợi Q và ngăn xếp S sau 2 thao tác sau:

S.push(Q.dequeue());
S.push(Q.dequeue());

Ghi chú:

- dequeue() là thao tác loại và trả về phần tử ở đầu hàng đợi.

b) Liệt kê các thao tác (trên Q, S và biến nếu cần) để tạo được ngăn xếp S có cấu trúc như sau:

a. [*j, n*]

b. [*j, i*]

Trong đó:

- Đỉnh ngăn xếp ở bên phải (nghĩa là phần tử cuối danh sách là đỉnh stack).
- *n* ở (a) là phần tử nằm trên cùng.
- *i* ở (b) là phần tử nằm trên cùng.

a.

Bước	Hàng đợi Q	Ngăn xếp S
Ban đầu	[i, j, m, n]	[]
Sau S.push(Q.dequeue()) (lấy i)	[j, m, n]	[i]
Sau S.push(Q.dequeue()) (lấy j)	[m, n]	[i, j]

b. Tạo S = [j, n] (n trên cùng)

a)

Bước	Hàng đợi Q	Ngăn xếp S	Ghi chú
Ban đầu	[i, j, m, n]	[]	
Q.dequeue() loại i	[j, m, n]	[]	bỏ i
Q.dequeue() → push j	[m, n]	[j]	
Q.dequeue() loại m	[n]	[j]	bỏ m
Q.dequeue() → push n	[]	[j, n]	n lên trên cùng

b)

Bước	Hàng đợi Q	Ngăn xếp S	Biến tạm	Ghi chú
Ban đầu	[i, j, m, n]	[]	-	
Q.dequeue() → lưu i	[j, m, n]	[]	tmp = i	
Q.dequeue() → push j	[m, n]	[j]	tmp = i	
push tmp vào stack	[m, n]	[j, i]	-	i lên trên cùng

Chương 6: Cây

Bài 1: Vẽ cây B bậc 3 và B bậc 4 được tạo ra bằng cách chèn dãy dữ liệu sau theo thứ tự:

92 24 6 7 11 8 22 4 5 16 19 20 78

Bậc 3	Bậc 4

Bài 2: Vẽ hai cây B bậc 3 được tạo ra bằng cách chèn dữ liệu lần lượt từ hai tập dưới đây. So sánh hai cây B để xác định xem thứ tự dữ liệu có tạo ra các cây B khác nhau hay không:

- Tập 1: 89, 78, 8, 19, 20, 33, 56, 44
- Tập 2: 44, 56, 33, 20, 19, 8, 78, 89

Bài 3: Tạo một cây B* bậc 5 từ dãy dữ liệu được chèn theo thứ tự sau:

92, 24, 6, 7, 11, 8, 22, 4, 5, 16, 19, 20, 78

Bài 4: Sử dụng ADT cây B (B-tree ADT), hãy tạo một cây B bậc 7 có 100 phần tử. Dùng trình tạo số ngẫu nhiên để tạo ngẫu nhiên các khóa nằm trong khoảng từ 1 đến 1000. Sau đó, tạo một giao diện người dùng điều khiển bằng menu, cho phép người dùng: chèn dữ liệu, xóa dữ liệu, truy xuất dữ liệu hoặc in cây ra màn hình.

Bài 5:

a) Lần lượt chèn các khóa sau [6, 4, 9, 2, 3, 5, 7, 1] vào một cây tìm kiếm nhị phân rỗng. Vẽ cây tìm kiếm nhị phân sau khi chèn hết các khóa trên.

b) Trình bày các bước để chèn đỉnh có khóa 8 vào cây thu được ở câu 1.

c) Cây thu được ở câu 2 có cân bằng hay không. Giải thích.

d) Trình bày các bước để xóa đỉnh có khóa 6 khỏi cây thu được ở câu 2.

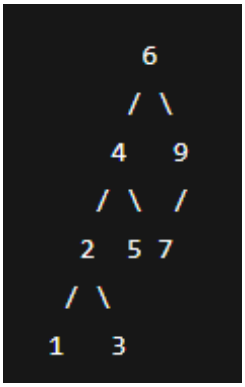
a) Chèn các khóa [6, 4, 9, 2, 3, 5, 7, 1] vào cây tìm kiếm nhị phân rỗng

Quy tắc chèn trong cây tìm kiếm nhị phân (BST):

Nếu khóa < nút hiện tại → chèn vào cây con bên trái

Nếu khóa > nút hiện tại → chèn vào cây con bên phải

Bắt đầu từ cây rỗng, lần lượt chèn:



6 → làm gốc

4 < 6 → chèn bên trái của 6

9 > 6 → chèn bên phải của 6

2 < 6 → trái → < 4 → trái → chèn bên trái của 4

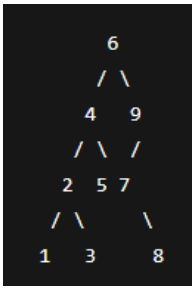
3 < 6 → trái → < 4 → trái → > 2 → phải → chèn bên phải của 2

5 < 6 → trái → > 4 → chèn bên phải của 4

7 > 6 → phải → < 9 → chèn bên trái của 9

1 < 6 → trái → < 4 → trái → < 2 → chèn bên trái của 2

b) Chèn đỉnh có khóa 8 vào cây ở câu a



Bắt đầu từ gốc 6:

8 > 6 → sang phải

8 < 9 → sang trái

8 > 7 → sang phải → chèn vào phải của 7

c) Cây thu được có cân bằng hay không? Giải thích.

Định nghĩa cây cân bằng (AVL Tree - độ cao cân bằng):

Một cây nhị phân được xem là cân bằng nếu:

- Với mọi nút, độ cao của cây con trái và phải không chênh lệch quá 1.

Phân tích độ cao:

- Gốc 6:
  - Trái (4) có chiều cao = 3 (nhánh 6→4→2→1)
  - Phải (9) có chiều cao = 3 (nhánh 6→9→7→8)

→ Chênh lệch độ cao = 0 → Cân bằng tại gốc

- Kiểm tra tiếp:
  - Nút 2: trái (1), phải (3) → chênh lệch 0
  - Nút 4: trái (2), phải (5) → độ cao hai bên là 2 → 1 → chênh lệch = 1
  - Nút 7: không có trái, phải (8) → chênh lệch = 1

Kết luận: Cây là cân bằng hợp lý, không vi phạm quy tắc cây cân bằng.

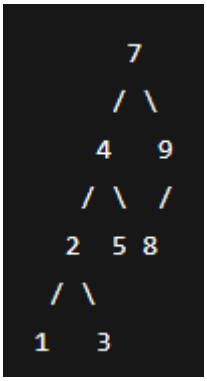
d) Xóa đỉnh có khóa 6 khỏi cây:

Nút 6 là gốc và có 2 con (trái: 4, phải: 9) → dùng cách thay thế bằng nút kế tiếp theo thứ tự (in-order successor) hoặc trước đó (in-order predecessor).

Chọn cách 1: Dùng in-order successor (nút nhỏ nhất bên phải → 7)

Các bước:

1. Tìm in-order successor của 6 → đi qua nút phải → tìm nút trái nhất của cây con phải → là 7
2. Thay giá trị 6 = 7
3. Xóa nút 7 (nút 7 chỉ có 1 con phải là 8 → thay thế bằng 8)



Bài 6: Viết chương trình cho phép người dùng nhập một dãy các số tự nhiên và trả lại kết quả trên màn hình là dãy số đã được sắp xếp theo thứ tự tăng dần. Sử dụng cấu trúc cây để lưu trữ.

```
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
def insert(root, key):
    if root is None:
        return Node(key)
    if key < root.key:
        root.left = insert(root.left, key)
    else:
        root.right = insert(root.right, key)
    return root
def inorder(root, result):
    if root is not None:
        inorder(root.left, result)
        result.append(root.key)
        inorder(root.right, result)
if __name__ == "__main__":
    data = input("Nhập dãy số tự nhiên (cách nhau bởi khoảng trắng): ")
    numbers = list(map(int, data.strip().split()))
    root = None
    for num in numbers:
        root = insert(root, num)
    sorted_list = []
    inorder(root, sorted_list)
    print("Dãy số đã sắp xếp tăng dần:")
    print(" ".join(str(x) for x in sorted_list))
```

Bài 7: Hãy tưởng tượng chúng ta có một danh sách sinh viên như trong bảng sau:

Tên	Email	Điểm
Alan Bush	ab123@gmail.com	89
Peter Cook	petercook12@hotmail.com	80
Alice Wonderland	wonderlandalice@gmail.com	99

a. Viết một hàm để tạo cây tìm kiếm nhị phân (Binary Search Tree) lưu trữ các sinh viên trong bảng trên, sử dụng điểm số làm khóa để sắp xếp cây.

b. Viết một hàm để in thông tin của sinh viên có điểm số cao nhất.

```
class Node:
    def __init__(self, name, email, score):
        self.name = name
        self.email = email
        self.score = score
        self.left = None
        self.right = None
class StudentBST:
    def __init__(self):
        self.root = None
    def insert(self, name, email, score):
        self.root = self._insert_recursive(self.root, name, email, score)
    def _insert_recursive(self, node, name, email, score):
        if node is None:
            return Node(name, email, score)
        if score < node.score:
            node.left = self._insert_recursive(node.left, name, email, score)
        else:
            node.right = self._insert_recursive(node.right, name, email, score)
        return node
    def print_highest_score(self):
        if self.root is None:
            print("Empty tree")
            return
        current = self.root
        while current.right is not None:
            current = current.right
        print("Student with highest score:")
        print(f"Name : {current.name}")
        print(f"Email: {current.email}")
        print(f"Score: {current.score}")
tree = StudentBST()
tree.insert("Alan Bush", "ab123@gmail.com", 89)
tree.insert("Peter Cook", "petercook12@hotmail.com", 80)
```

```
tree.insert("Alice Wonderland", "wonderlandalice@gmail.com", 99)
tree.print_highest_score()
```